SAI VIDYA INSTITUTE OF TECHNOLOGY

BANGLORE -560064, KARNATAKA

Project Report
on

"PING HUB"

BACHELOR OF ENGINEERING

Submitted By

Vaibhav B K	**AF05072305**
Keerthana Bhat	**AF05054480**
Bhavana Prakash Naik	**AF05060891**
Nikhita B	**AF05054505**

ABSTRACT

The rapid advancement of communication technologies has significantly transformed the way individuals interact and exchange information in both personal and professional environments. In recent years, web-based communication platforms have gained immense popularity due to their accessibility, flexibility, and ease of use. Messaging applications have become an essential part of daily life, enabling instant communication across geographical boundaries. However, many existing systems are complex, resource-intensive, and not easily customizable for academic or small-scale purposes. This has created a need for a simplified yet effective communication system that demonstrates real-world application development concepts.

The Room-Based Chat Application developed in this project is a web-based system designed to facilitate structured communication among users. The application allows users to join specific chat rooms and interact through text messages, images, files, and audio messages. By organizing communication within rooms, the system ensures clarity and structured interaction, making it suitable for group discussions, academic collaboration, and small organizational use.

One of the key features of the system is the implementation of secure access control through unique room access keys. Each chat room is assigned a specific key, which must be entered by users to gain access. This mechanism ensures that only authorized users can participate in private conversations, thereby maintaining the confidentiality and integrity of communication.

In addition to access control, the system incorporates data security through the use of AES (Advanced Encryption Standard) encryption. Messages are encrypted before being stored in the database and decrypted when displayed to the user. This ensures that sensitive information remains protected even if unauthorized access to the database occurs. The integration of encryption enhances the overall security of the system and demonstrates the practical application of cryptographic techniques in web development.

The backend of the application is developed using the Spring Boot framework, which provides a robust and scalable environment for handling business logic, database interaction, and request processing. The frontend is designed using HTML, CSS, JavaScript, and Thymeleaf, ensuring a responsive and user-friendly interface. MySQL is used as the database to store user details, chat rooms, messages, and access information.

The system follows a modular architecture, dividing the application into different layers such as presentation, business logic, and data access. This approach improves maintainability, scalability, and ease of development. Each module is designed to perform a specific function, ensuring a clear separation of concerns and efficient system operation.

The project also emphasizes usability and performance by providing a simple interface that allows users to navigate easily between chat rooms and perform actions without complexity. Multimedia support further enhances the user experience by enabling rich communication beyond text-based interaction.

Although the current implementation does not include real-time communication features such as WebSocket, it successfully simulates an interactive messaging environment suitable for demonstration and learning purposes. The system serves as a foundation for future enhancements, including real-time messaging, mobile application development, push notifications, and advanced security features.

In conclusion, the Room-Based Chat Application provides a practical and efficient solution for structured communication while demonstrating essential software development concepts such as authentication, database management, encryption, and user interface design. The project not only fulfills its objectives but also provides a strong foundation for developing advanced communication systems in the future.

PING HUB

Table of Contents

1 **Introduction**

2 **Objectives of the Project**

3 **Scope of the Project**

4 **Feasibility Study**

5 **System Design**

6 **Existing application**

7 **Proposed application**

8. **ER Diagram**

9. **Screenshots of database**

10. **Coding and your application screenshots**

11. **Conclusion**

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Project

In the present digital age, communication technologies have evolved significantly, transforming the way people interact and exchange information. Messaging applications have become an integral part of everyday life, enabling instant communication across geographical boundaries. Platforms such as WhatsApp, Telegram, and Messenger have set high standards for real-time interaction, multimedia sharing, and user-friendly interfaces.

With the increasing demand for communication systems, there is a growing need for developing customized chat applications that are lightweight, secure, and adaptable to specific requirements. This project focuses on designing and developing a Room-Based Chat Application using Spring Boot that provides essential messaging functionalities in a structured and efficient manner.

The proposed system enables users to join chat rooms and communicate with other users through text messages, images, files, and audio recordings. The system is designed to simulate the behavior of modern chat applications while maintaining simplicity in design and implementation.

In addition to providing basic communication functionality, the system also focuses on maintaining a balance between simplicity and security. Modern communication platforms are often built using highly complex architectures that are difficult to understand and implement for academic purposes. Therefore, this project aims to simplify these concepts while still incorporating essential features such as secure communication, structured interaction, and multimedia support.

The design of this application emphasizes modular development, where each component performs a specific role. This approach not only improves the clarity of the system but also makes it easier to debug, maintain, and extend in the future. By dividing the system into logical modules such as user management, chat room handling, and messaging, the application achieves a well-organized structure that reflects real-world software engineering practices.

1.2 Background of the Project

Traditional communication methods such as emails and SMS are gradually being replaced by instant messaging systems due to their speed, convenience, and flexibility. Modern chat applications offer features such as:

- Real-time messaging
- Multimedia sharing
- Group communication
- Message status indicators
- Data synchronization

However, many existing systems are either too complex, resource-intensive, or not suitable for academic or small-scale applications. Additionally, most commercial chat applications are closed-source and cannot be easily customized.

This creates an opportunity to develop a custom chat application that:

- Meets specific requirements
- Provides essential communication features
- Is easy to understand and implement
- Can be extended for future improvements

With the rapid advancement of internet technologies, communication has evolved from traditional methods to highly interactive digital platforms. Messaging applications have become an essential part of everyday life, allowing users to exchange information instantly. However, most of these applications rely on advanced technologies and large-scale infrastructure, making them unsuitable for small-scale or academic implementations.

In an educational context, it is important to understand the underlying principles of such systems rather than just using them. This project provides an opportunity to explore how communication systems are designed and implemented using modern programming frameworks. By developing a chat application from scratch, the project helps in understanding key concepts such as request handling, data storage, and user interaction.

1.3 Problem Statement

Despite the availability of numerous communication platforms, there is a need for a simple, customizable, and educational chat application that demonstrates real-world software development concepts.

The key problems identified are:

- Lack of simple and customizable chat systems for academic use
- Complex architecture of existing applications
- Limited understanding of real-time communication systems among students
- Difficulty in integrating messaging features in beginner-level projects

Therefore, this project aims to develop a Room-Based Chat Application that addresses these issues by providing a simplified and structured communication platform.

1.4 Purpose of the Project

The primary purpose of this project is to design and develop a web-based chat application that allows users to communicate efficiently within chat rooms.

The project also aims to provide a foundation for future enhancements. While the current implementation includes essential features, it can be extended to include advanced functionalities such as real-time messaging, notifications, and mobile support. This makes the project not only a learning tool but also a base for further development.

The system aims to:

- Provide a platform for group communication
- Demonstrate the implementation of messaging systems
- Integrate backend and frontend technologies
- Enhance user interaction through multimedia support

1.5 Features of the System

The chat application includes several features that enhance user experience and functionality:

1. User Authentication

- OTP-based login system
- Secure session management

2. Chat Room Management

- Create and join chat rooms
- Room-based communication

3. Messaging System

- Send text messages
- Display messages instantly (page refresh based)

4. Media Sharing

- Upload and send images
- Share files and documents
- Record and send audio messages

5. Message Control

- Soft delete (message visible as deleted)
- Permanent delete

6. User Interface

- Attractive and responsive design
- Easy navigation

1.6 Technologies Used

The project is developed using the following technologies:

Backend:

- Spring Boot
- Java

Frontend:

- HTML
- CSS
- Thymeleaf

Database:

- MySQL

Tools:

- Spring Tool Suite (STS)
- Maven

1.7 Advantages of the System

- Easy to use and understand
- Lightweight and fast
- Customizable
- No installation required
- Supports multimedia communication

1.8 Applications of the System

The system can be used in:

- Educational institutions
- Group discussions
- Small organizations
- Team collaboration

1.9 Importance of Secure Communication Systems

In modern computing environments, data security plays a crucial role in communication systems. With the increasing number of cyber threats, unauthorized access, and data breaches, it has become essential to protect user communication. Messaging applications are often targeted by attackers due to the sensitive nature of the information exchanged.

The implementation of encryption algorithms such as AES ensures that even if the database is compromised, the actual message content remains unreadable. This project integrates encryption at the application level, thereby enhancing data confidentiality and protecting user privacy.

Furthermore, secure access mechanisms such as room access keys prevent unauthorized users from entering private communication channels. This ensures controlled communication and maintains the integrity of chat rooms.

1.10 Role of Web-Based Applications

Web-based applications provide significant advantages over traditional desktop applications. They are platform-independent, easily accessible, and do not require installation. The chat application developed in this project operates entirely within a web browser, making it convenient for users to access from any device with an internet connection.

1.11 Need for Custom Chat Applications

In recent years, communication platforms have become essential tools for interaction in various domains such as education, business, and social networking. While many commercial chat applications are available, they are often designed for large-scale usage and include features that are not required for smaller or academic systems.

There is a growing need for custom-built chat applications that are lightweight, easy to understand, and tailored to specific requirements. Such systems help developers gain practical knowledge and allow organizations to create solutions that match their needs.

This project addresses this need by providing a simplified chat system that demonstrates essential communication features while maintaining flexibility for future improvements.

1.12 Challenges in Developing Chat Applications

Developing a chat application involves several challenges, including:

- Managing multiple users simultaneously
- Ensuring data security and privacy
- Handling multimedia data efficiently
- Designing a responsive and user-friendly interface

This project attempts to overcome these challenges by implementing structured design, encryption mechanisms, and efficient database management.

1.13 Conclusion of Introduction

The Room-Based Chat Application is a practical implementation of a communication system that demonstrates key software development concepts. It provides a foundation for understanding how messaging platforms work and can be extended further to include advanced features such as real-time communication and mobile support.

CHAPTER 2

OBJECTIVES OF THE PROJECT

2.1 Primary Objective

The primary objective of this project is to design and develop a Room-Based Chat Application that enables users to communicate effectively through structured chat rooms using a web-based platform.

2.2 Secondary Objectives

1. To Develop a User-Friendly Chat Interface

The system aims to provide an intuitive and easy-to-use interface that allows users to send and receive messages without complexity.

2. To Implement Secure Authentication

The project includes an OTP-based login system to ensure secure user authentication and prevent unauthorized access.

3. To Enable Room-Based Communication

Users can join specific chat rooms, making communication more organized and structured.

4. To Support Multimedia Messaging

The application allows users to send:

- Images
- Files
- Audio messages

5. To Implement Message Management

Users can:

- Delete messages temporarily (soft delete)
- Permanently remove messages

6. To Integrate Frontend and Backend

The project demonstrates how frontend UI interacts with backend logic using Spring Boot.

7. To Provide Learning Experience

The project helps students understand:

- MVC architecture
- Database interaction
- File handling
- User session management

8. To Build Scalable Architecture

The system is designed in a modular way so it can be extended with new features in the future.

2.3 Advanced Objectives

1. To Implement Data Encryption

The system aims to protect sensitive communication by encrypting messages using AES encryption before storing them in the database.

2. To Design Secure Access Control

The system ensures that only authorized users can enter specific chat rooms using a unique access key mechanism.

3. To Handle Multimedia Efficiently

The system supports uploading and displaying multimedia content while maintaining performance and usability.

2.4 Functional Objectives

The functional objectives define the specific operations that the system must perform. These include user authentication, room access control, message handling, and media sharing.

Each function is carefully designed to ensure smooth interaction between users and the system. The messaging functionality, for example, ensures that messages are transmitted, stored, and displayed efficiently.

2.5 Non-Functional Objectives

Non-functional objectives focus on system performance and usability. These include:

- System reliability
- Fast response time
- Data security
- User-friendly interface

These objectives ensure that the system not only functions correctly but also provides a good user experience.

CHAPTER 3

SCOPE OF THE PROJECT

3.1 Introduction to Project Scope

The scope of the Room-Based Chat Application defines the functional boundaries, technical capabilities, and limitations of the system. It describes what the system is designed to perform, the range of its operations, and the areas where it can be applied.

The project is primarily focused on building a **web-based communication system** that allows users to interact through chat rooms. It is designed as a learning-oriented system that demonstrates the working principles of modern messaging platforms while maintaining simplicity and usability.

The scope includes communication features, data management, user interaction, and media sharing functionalities, but does not extend to highly complex real-time systems or large-scale enterprise messaging platforms.

3.2 Functional Scope

3.2.1 User Authentication and Management

The system includes a secure authentication mechanism where users can log in using an OTP-based system. This ensures that only authorized users can access the application.

Functional capabilities:

- User login with OTP verification
- Session management
- User identity tracking within chat rooms

3.2.2 Chat Room Management

The application supports room-based communication where users can join different chat rooms.

Features include:

- Viewing available chat rooms
- Joining a chat room
- Displaying messages specific to a room
- Managing room-based communication flow

3.2.3 Messaging System

The core functionality of the system is message exchange between users.

The system supports:

- Sending text messages
- Displaying messages in chat interface
- Maintaining message history
- Associating messages with users and rooms

3.2.4 Media Sharing

The application extends beyond text communication and allows users to share multimedia content.

Supported media:

- Images (photo sharing)
- Files (documents, PDFs)
- Audio recordings (voice messages)

This enhances communication and makes the system similar to modern messaging applications.

3.2.5 Message Management

The system provides users control over their messages through deletion options.

Types of deletion:

- Soft delete: Message appears as “This message was deleted”
- Permanent delete: Message is removed from database

3.2.6 User Interface Scope

The application includes:

- Attractive chat interface
- Responsive layout
- Easy navigation
- Message display formatting

3.3 Technical Scope

3.3.1 System Architecture

The system follows a **3-tier architecture**:

1. Presentation Layer (Frontend UI)
2. Business Logic Layer (Spring Boot)
3. Data Layer (MySQL Database)

This separation improves maintainability and scalability.

3.3.2 Backend Capabilities

The backend handles:

- Message processing
- File upload handling
- Database operations
- Business logic

3.3.3 Database Scope

The system stores:

- User details
- Chat rooms
- Messages
- Media file references

The database ensures data consistency and efficient retrieval.

3.3.4 Integration Scope

The system can be extended to integrate with:

- Email services (OTP sending)
- Cloud storage (for media files)
- Notification services

3.4 User-Based Scope

3.4.1 Users

Users can:

- Login using OTP
- Join chat rooms
- Send messages
- Upload media
- Delete messages

3.4.2 Administrator

Admin can:

- Manage chat rooms
- Monitor user activity
- Control system behavior

3.5 Operational Scope

The system can be used in:

- Educational institutions
- Group discussions
- Team communication
- Project collaboration

3.6 Security Scope

The system ensures:

- Secure login
- Session handling
- Controlled access
- Safe file upload

3.7 Future Scope

The system can be extended with:

- Real-time messaging (WebSocket)
- Emoji support
- Message status (sent/delivered/read)
- Notifications
- Mobile application

3.10 Security Scope (DETAILED)

The security scope of the system focuses on protecting both user data and communication channels. The system integrates encryption techniques and access control mechanisms to ensure that data is not exposed to unauthorized users.

Key security features include:

- AES encryption for message confidentiality
- Unique room access keys for authorization
- Controlled database access
- Secure session handling

3.11 Performance Scope

The system is designed to handle moderate levels of user interaction efficiently. The use of optimized database queries ensures that messages are retrieved quickly. Additionally, the system avoids unnecessary data duplication, thereby improving performance.

3.12 Functional Scope in Detail

The functional scope of the system includes all operations related to user interaction and system behavior. The system allows users to perform multiple actions such as joining chat rooms, sending messages, uploading files, and deleting messages.

Each function is designed to ensure accuracy and efficiency. For example, message handling ensures that data is processed and displayed without delay.

3.13 Technical Limitations

While the system is designed to be efficient, it has certain technical limitations such as:

- Dependency on internet connectivity
- Limited scalability for large users
- No real-time communication

These limitations can be addressed in future versions.

3.14 Summary of Scope

The scope of the Room-Based Chat Application includes:

- Messaging functionality
- Media sharing
- User authentication
- Database management
- UI design

It provides a complete foundation for building a communication system while allowing future enhancements.

CHAPTER 4

FEASIBILITY STUDY

4.1 Introduction to Feasibility Study

A feasibility study evaluates whether the proposed system can be successfully developed and implemented within the given constraints.

It analyzes:

- Technical feasibility
- Economic feasibility
- Operational feasibility
- Schedule feasibility

4.2 Technical Feasibility

4.2.1 Availability of Technology

The system uses:

- Java (Spring Boot)
- MySQL
- HTML, CSS, JavaScript

These technologies are:

- Widely available
- Easy to use
- Well-documented

4.2.2 Development Complexity

The project follows a modular structure which:

- Simplifies development
- Enables independent testing

- Supports incremental improvement

4.2.3 Integration Feasibility

The system can easily integrate with:

- Email services
- Cloud storage
- APIs

4.2.4 Performance Feasibility

The system supports:

- Efficient database queries
- Fast message processing
- Stable performance for small-scale usage

4.3 Economic Feasibility

4.3.1 Development Cost

The project is cost-effective because:

- Uses open-source tools
- Requires no licensing cost
- Can be developed on personal computers

4.3.2 Operational Cost

Low operational cost due to:

- Minimal hardware requirements
- Free development tools
- Basic hosting requirements

4.3.3 Cost-Benefit Analysis

Benefits include:

- Learning experience
- Practical implementation
- Low cost with high value

4.4 Operational Feasibility

4.4.1 User Acceptance

The system is:

- Easy to use
- Simple interface
- Requires minimal training

4.4.2 System Usability

Users can easily:

- Navigate UI
- Send messages
- Upload files

4.5 Schedule Feasibility

The project can be completed within academic timeline due to:

- Clear structure
- Defined modules
- Reusable components

4.6 Legal Feasibility

The system:

- Does not violate data privacy
- Stores user data securely
- Uses ethical practices

4.7 Social Feasibility

The system promotes:

- Easy communication
- Collaboration
- Digital interaction

4.8 Resource Feasibility

Hardware Requirements

- Computer
- Internet connection

Software Requirements

- Java
- MySQL
- Spring Boot

4.9 Feasibility Conclusion

The project is:

✔	Technically	feasible
✔	Economically	viable

✔ Operationally practical

✔Easy to implement

4.10 Risk Analysis

The development of the system involves certain risks such as:

- Data security risks
- System performance issues
- Incorrect user input
- Server downtime

These risks are minimized through proper validation, encryption, and efficient coding practices.

4.11 Summary

The feasibility study confirms that the project can be successfully developed and deployed with available resources and technologies.

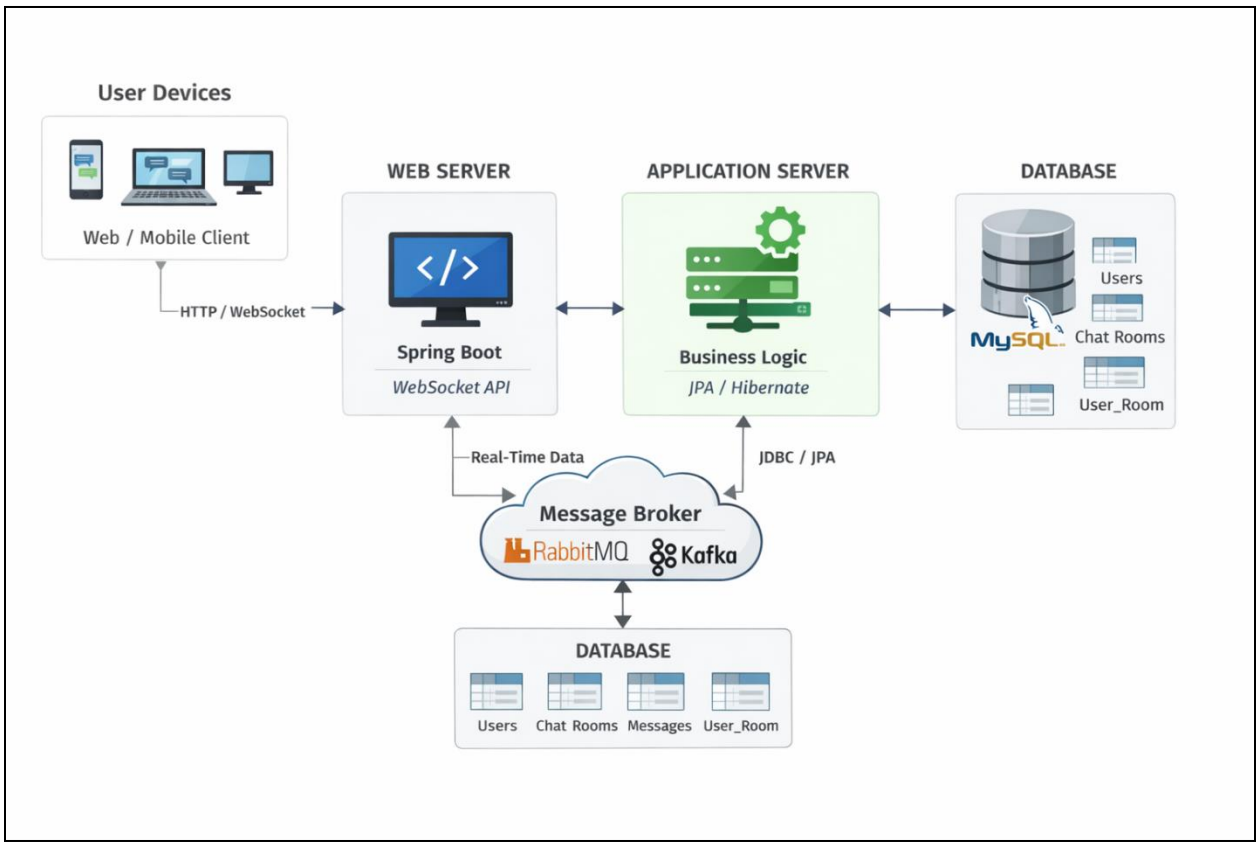
CHAPTER 5

SYSTEM DESIGN

5.1 Introduction

System design is a critical phase in software development that defines the architecture, components, modules, interfaces, and data flow of the system. The Room-Based Chat Application is designed using a modular and layered architecture to ensure scalability, maintainability, and security.

The system incorporates modern design principles such as separation of concerns, modularity, and data abstraction. It integrates user authentication, chat room management, messaging services, and encryption mechanisms into a cohesive structure.



5.2 Architectural Design

The application follows a **three-tier architecture model**, which includes:

1. Presentation Layer

This layer consists of:

- HTML pages
- CSS styling
- JavaScript functionality
- Thymeleaf templates

It is responsible for:

- Displaying chat interface
- Taking user input
- Sending requests to backend

2. Business Logic Layer

This layer is implemented using Spring Boot.

It handles:

- Message processing
- User authentication
- Room access control
- Encryption and decryption

3. Data Layer

The data layer interacts with MySQL database.

It stores:

- User information
- Chat room details
- Messages
- Access permissions

5.3 Module Design

□ 1. User Module

Handles:

- OTP login
- Session management
- User identification

□ 2. Chat Room Module

This module is responsible for:

- Creating rooms
- Generating access keys
- Managing user access

□ Access Key Algorithm

When admin creates room:

```
Key = UUID.random().substring(0,6)
```

Example:

```
AB12CD
```

□ 3. Access Control Mechanism

From your document :

The system implements a **secure access mechanism**:

- Each room has unique key
- User must enter key
- System validates key
- If correct → access granted
- Access stored in database

4. Messaging Module

Handles:

- Sending messages
- Receiving messages
- Displaying chat history

5. Encryption Module

Based on your document

The system uses AES encryption for message security.

5.4 Data Flow Design

Flow:

1. User logs in
2. User selects room
3. System checks access:
    - If general chat → allow
    - Else → ask key
4. User enters key
5. System validates key

6. Access granted and stored
7. User sends message
8. Message encrypted
9. Stored in database
10. Retrieved and decrypted

5.5 Security Design

Security is implemented at multiple levels:

1. Authentication Security

- OTP login

2. Room Security

- Unique access key
- Controlled entry

3. Data Security

- AES encryption

5.6 Database Design

Tables used:

- User
- ChatRoom
- Message
- Room_Access

5.7 Special Feature: General Chat

The system includes a special room:

 "General Chat"

Rules:

- No key required
- Open for all users

5.8 System Workflow

1. Admin creates room
2. Key generated automatically
3. User joins room using key
4. Access stored
5. User chats securely

5.9 Detailed Working of AES Encryption Algorithm

AES (Advanced Encryption Standard) is a symmetric encryption algorithm widely used for securing sensitive data.

Working Process:

1. Plain text message is taken as input
2. Secret key is applied
3. Message is converted into cipher text
4. Cipher text is stored in database

Decryption Process:

1. Cipher text is retrieved
2. Same key is applied
3. Original message is obtained

5.10 Advantages of AES in This Project

- Strong encryption standard
- Fast performance
- Secure against attacks
- Widely accepted algorithm

5.11 Room Access Key Generation Algorithm

The system uses a random key generation mechanism:

Steps:

1. Generate random UUID
2. Extract substring
3. Convert to uppercase
4. Assign as room key

Example:

```
Generated Key → AB12CD
```

5.12 Access Validation Algorithm

1. User enters key
2. System compares entered key with stored key
3. If matched → access granted
4. Else → access denied

5.13 Data Flow Explanation

The system follows a structured data flow:

- Input from user
- Processing in controller

- Business logic execution
- Database interaction
- Response sent to UI

5.14 Detailed System Workflow Analysis

The Room-Based Chat Application follows a structured workflow that ensures smooth communication between users while maintaining security and data integrity. The workflow begins with user authentication and continues through room access validation, message exchange, and data storage.

When a user logs into the system, a session is created and maintained throughout the interaction. This session ensures that the user identity is preserved while navigating between different pages. Once authenticated, the user is presented with available chat rooms.

If the user selects a general chat room, access is granted immediately. However, for protected chat rooms, the system prompts the user to enter an access key. This key acts as a security mechanism to restrict unauthorized entry.

After entering the correct key, the system validates the input and grants access. The user's access is stored, allowing future entry without repeated validation. Once inside the chat room, the user can send and receive messages.

Each message undergoes encryption before being stored in the database. This ensures that sensitive communication remains secure even if unauthorized access to the database occurs. When messages are retrieved, they are decrypted and displayed in readable format.

This workflow ensures that the system maintains a balance between usability and security while providing a seamless user experience.

5.15 Detailed Module Interaction

The system consists of multiple modules that interact with each other to perform various operations. These modules include the user module, chat room module, messaging module, and encryption module.

The user module handles authentication and session management. Once the user is authenticated, the chat room module takes over to manage room access and validation. The messaging module

is responsible for sending and retrieving messages, while the encryption module ensures data security.

The interaction between these modules is coordinated through the controller layer, which acts as an intermediary between the frontend and backend. This modular design enhances maintainability and allows independent modification of components without affecting the entire system.

5.16 Detailed Security Architecture

Security is one of the most important aspects of the system. The application implements multiple layers of security to protect user data and communication.

The first layer is authentication, where users must verify their identity through OTP. The second layer is authorization, where access to chat rooms is controlled using unique keys. The third layer is data security, where messages are encrypted before storage.

The encryption mechanism ensures that even if an attacker gains access to the database, the data remains unreadable. The access control mechanism prevents unauthorized users from entering restricted chat rooms.

Together, these security measures create a robust system that protects both user identity and communication data.

5.17 Internal Working of Message Processing

The message processing mechanism involves several steps that ensure reliable communication. When a user sends a message, the input is first validated to ensure it is not empty or invalid. The message is then passed to the backend for processing.

Before storing the message, encryption is applied to protect its content. The encrypted message is stored in the database, and when retrieved, it is decrypted and displayed to the user.

5.18 Detailed Explanation of Access Control

Access control is implemented to restrict entry to chat rooms. Each room is assigned a unique key, which acts as a password. Users must enter this key to gain access.

Once verified, the system records the access, allowing the user to enter the room without repeated verification. This mechanism ensures both security and convenience.

5.19 System Scalability Considerations

Although the system is designed for moderate usage, scalability is considered in its design. The modular structure allows additional features to be integrated without major changes.

The system can be extended to handle larger user bases by optimizing database queries and implementing real-time communication technologies.

CHAPTER 6

EXISTING SYSTEM

6.1 Introduction to Existing System

In today's digital environment, communication systems have evolved significantly, with numerous applications providing instant messaging capabilities. Existing chat applications such as WhatsApp, Telegram, and Facebook Messenger are widely used for communication across personal and professional domains. These systems provide advanced features such as real-time messaging, multimedia sharing, encryption, and group communication.

However, despite their popularity, these systems are not always suitable for academic projects or customized implementations due to their complexity and closed architecture.

6.2 Description of Existing Systems

Existing messaging systems operate on highly complex architectures that include:

- Real-time communication using WebSockets
- End-to-end encryption protocols
- Distributed server systems
- Cloud storage integration
- Cross-platform compatibility

These systems provide seamless communication but are difficult to replicate for learning purposes.

6.3 Features of Existing Systems

Modern chat applications provide:

- Instant messaging
- Voice and video calls
- Multimedia sharing
- Message encryption
- Group chats
- Message reactions
- Online/offline status

6.4 Limitations of Existing Systems

Despite their advanced features, existing systems have several limitations:

1. Complexity

The architecture of modern chat applications is highly complex and difficult to understand for beginners.

2. Lack of Customization

Most applications are closed-source and do not allow modification according to user requirements.

3. High Resource Usage

They require:

- High bandwidth
- Advanced servers
- Large storage capacity

4. Not Suitable for Learning

Students cannot easily:

- Understand internal working
- Modify features
- Implement new functionalities

5. Security Complexity

While they provide encryption, the implementation of these systems is complex and not easily understandable for academic purposes.

6.5 Need for Proposed System

Due to the limitations mentioned above, there is a need for a system that:

- Is simple and easy to understand
- Can be customized easily
- Demonstrates real-world concepts
- Supports essential messaging features

Comparison Table

Feature	Existing System	Proposed System
Complexity	High	Low
Customization	Limited	High
Security	Advanced	Moderate (AES-based)
Learning	Difficult	Easy

6.6 Summary

The existing systems, although powerful, are not suitable for learning and small-scale implementation. This motivates the development of a simplified chat application that retains essential features while being easy to develop and understand.

CHAPTER 7

PROPOSED APPLICATION

7.1 Introduction to Proposed System

The proposed system is a **Room-Based Chat Application** developed using Spring Boot that provides a simplified yet effective communication platform. It is designed to overcome the limitations of existing systems by offering a lightweight and customizable solution.

7.2 Overview of Proposed System

The system allows users to:

- Login using OTP
- Join chat rooms
- Send messages
- Share images, files, and audio
- Delete messages
- Access rooms using secure keys

7.3 Key Features of Proposed System

1. Room-Based Communication

Users communicate within specific rooms, ensuring organized interaction.

2. Secure Room Access Mechanism

Based on your document :

- Each room has a unique access key
- Only users with the correct key can enter
- Access is stored for future use

3. General Chat Room

- Open to all users
- No key required
- Acts as a public communication channel

4. AES Encryption for Messages

Based on your document :

- Messages are encrypted before storing
- Decrypted when displayed
- Ensures data security

5. Multimedia Messaging

Users can send:

- Images
- Files
- Audio messages

6. Message Deletion

- Soft delete
- Permanent delete

7.4 Advantages of Proposed System

1. Simplicity

The system is easy to understand and implement.

2. Security

- AES encryption
- Controlled room access

3. Flexibility

- Customizable features
- Extendable architecture

4. Cost Efficiency

- Uses open-source technologies
- Low development cost

7.5 Working of Proposed System

Step-by-step process:

1. Admin creates chat room
2. System generates unique key
3. User enters key to access room
4. Access is stored in database
5. User sends message
6. Message encrypted using AES
7. Stored in database
8. Retrieved and decrypted for display

7.6 Efficiency of Proposed System

The proposed system is efficient in terms of performance and usability. It minimizes resource usage while providing essential communication features.

The use of optimized database operations ensures quick message retrieval, and the structured design improves system reliability.

7.7 Reliability of the System

The system is designed to provide consistent performance under normal conditions. Error handling mechanisms ensure that unexpected issues do not affect overall functionality.

7.8 Summary

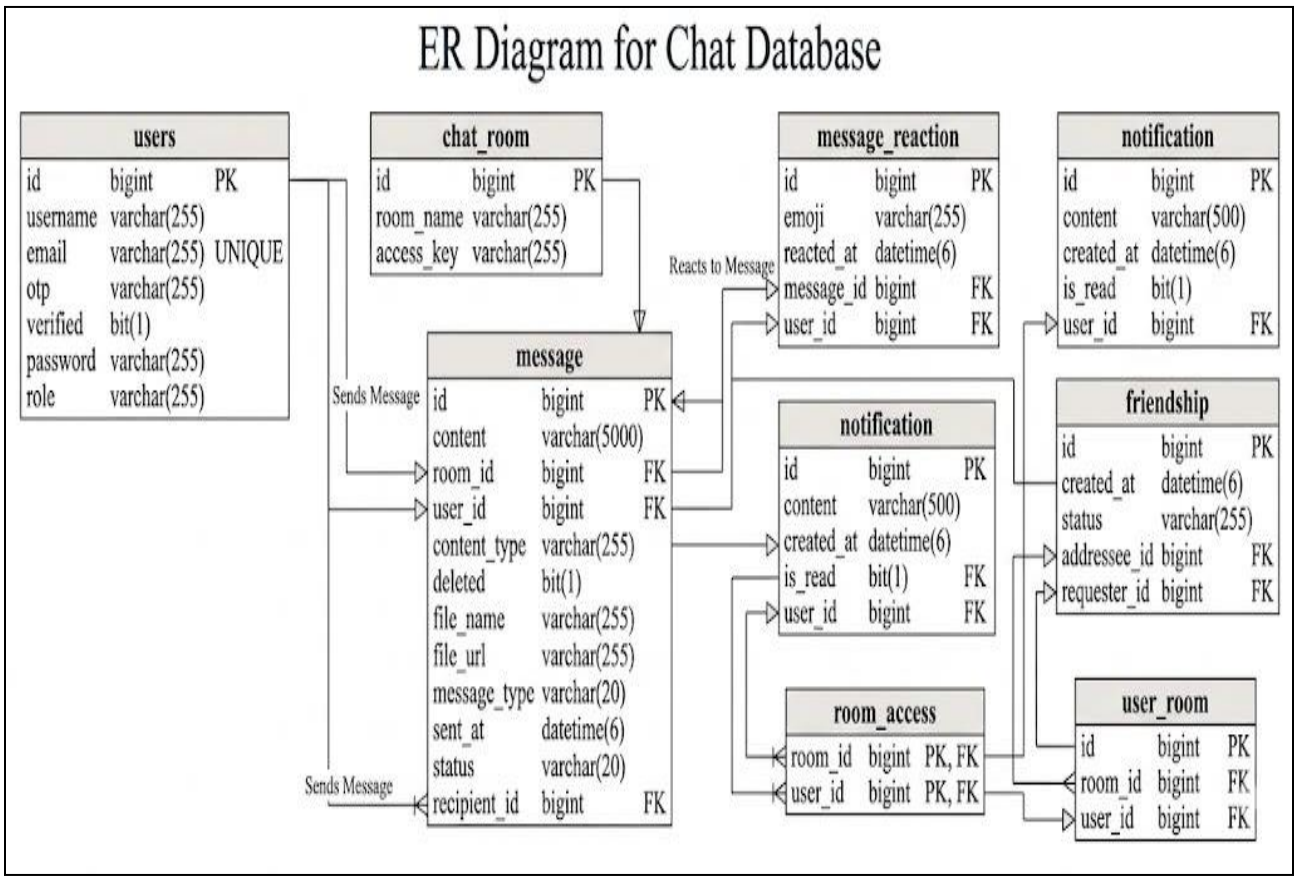
The proposed system provides a simplified yet effective communication platform with enhanced security and usability, making it suitable for academic and small-scale applications.

CHAPTER 8

ER DIAGRAM

8.1 Introduction to ER Diagram

The Entity Relationship (ER) diagram represents the structure of the database and the relationships between different entities in the system.



8.2 Entities in the System

1. User

Attributes:

- id
- username
- email

2. ChatRoom

Attributes:

- id
- roomName
- accessKey

3. Message

Attributes:

- id
- content (encrypted)
- sentAt
- status

4. Room Access

Represents relationship between:

- User
- ChatRoom

8.3 Relationships

1. User → Message

- One user can send multiple messages

2. ChatRoom → Message

- One room contains multiple messages

3. User ↔ ChatRoom

- Many-to-many relationship
- Managed through access table

8.4 ER Diagram Explanation

- Users interact with chat rooms
- Chat rooms contain messages
- Messages belong to both user and room
- Access control ensures security

8.5 Database Logic

The system ensures:

- Data integrity
- Relationship consistency

- Efficient storage

8.6 Role of ER Diagram

- Helps in database design
- Ensures proper structure
- Avoids redundancy

8.7 Detailed Relationship Explanation

The relationship between entities ensures that:

- Each message is linked to a user
- Each message belongs to a room
- Access control is maintained

This prevents data inconsistency and ensures structured storage.

CHAPTER 9

DATABASE SCREENSHOTS AND EXPLANATION

9.1 Introduction to Database Design

The database plays a crucial role in the Room-Based Chat Application as it stores all the persistent data required for the system to function. The application uses a relational database (MySQL) to store user information, chat room details, messages, and access control data.

The database is designed to ensure:

- Data integrity
- Efficient storage
- Fast retrieval
- Security

9.2 Database Tables Overview

The system consists of the following main tables:

1. User Table
2. ChatRoom Table
3. Message Table
4. Room_Access Table

```
mysql> select*from chat_room;
+----+-----------+-------------+
| id | room_name | access_key |
+----+-----------+-------------+
|  4 | room 1    | NULL        |
|  5 | room 2    | NULL        |
| 12 | room 3    | 1D025C      |
+----+-----------+-------------+
3 rows in set (0.00 sec)
```

```
mysql> select*from friendship;
+----+----------------------------+----------+--------------+--------------+
| id | created_at                 | status   | addressee_id | requester_id |
+----+----------------------------+----------+--------------+--------------+
|  1 | 2026-04-13 12:56:39.890935 | PENDING  |            1 |           12 |
|  2 | 2026-04-13 12:56:54.952125 | PENDING  |           11 |           12 |
|  3 | 2026-04-14 09:26:56.731770 | ACCEPTED |            4 |           12 |
|  4 | 2026-04-14 09:36:31.843290 | PENDING  |            3 |           12 |
|  5 | 2026-04-14 09:37:42.165666 | PENDING  |            2 |           12 |
+----+----------------------------+----------+--------------+--------------+
5 rows in set (0.01 sec)
```

```
mysql> select*from message;
+----+-----------------------+---------+---------+----------------------------+---------+-------------+-----------------------------------------------------------+-----------------------------------------------------------+-------------+---------+-----------+
| id | content               | room_id | user_id | sent_at                    | deleted | content_type | file_name                                                 | file_url                                                  | message_type | status  | recipient_id |
+----+-----------------------+---------+---------+----------------------------+---------+-------------+-----------------------------------------------------------+-----------------------------------------------------------+-------------+---------+-----------+
|  1 | hi                    |    NULL |    NULL | NULL                       | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | NULL    |      NULL |
|  2 | hi guys               |    NULL |    NULL | NULL                       | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | NULL    |      NULL |
|  3 | hi                    |    NULL |    NULL | NULL                       | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | NULL    |      NULL |
|  4 | hi guys               |    NULL |    NULL | NULL                       | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | NULL    |      NULL |
|  5 | hi                    |    NULL |    NULL | NULL                       | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | NULL    |      NULL |
|  6 | hi guys               |    NULL |    NULL | NULL                       | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | NULL    |      NULL |
|  7 | hi guys               |    NULL |    NULL | NULL                       | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | NULL    |      NULL |
|  8 | hi                    |    NULL |    NULL | NULL                       | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | NULL    |      NULL |
| 13 | hi                    |       4 |       2 | 2026-04-08 13:18:11.366501 | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | DELIVERED |      NULL |
| 15 | hello                 |       4 |       4 | 2026-04-08 13:31:56.903874 | 0x00    | NULL        | NULL                                                      | NULL                                                      | NULL        | DELIVERED |      NULL |
| 16 | This message was deleted |    4 |       2 | 2026-04-09 10:25:47.142426 | 0x01    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 17 | NULL                  |       4 |       2 | 2026-04-09 10:27:53.824527 | 0x00    | image/png   | Screenshot 2026-03-22 121313.png                          | /uploads/72837a4e-52bc-4fc1-a20f-1ec1b1c0e4ca.png         | IMAGE       | DELIVERED |      NULL |
| 18 | hello                 |       4 |       2 | 2026-04-09 10:28:50.848096 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 19 | ?                     |       4 |       2 | 2026-04-09 10:29:31.957086 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 20 | This message was deleted |    4 |       2 | 2026-04-09 10:40:35.865697 | 0x01    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 21 | This message was deleted |    4 |       3 | 2026-04-09 10:41:46.775517 | 0x01    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 22 | This message was deleted |    4 |       2 | 2026-04-09 12:07:46.437811 | 0x01    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 23 | NULL                  |       4 |       2 | 2026-04-09 12:07:18.711271 | 0x00    | application/pdf | Keerthana_Bhat_Resume (1).pdf                             | /uploads/03e8829a-cc92-485e-8e2e-9f3fe42cb63e.pdf         | FILE        | DELIVERED |      NULL |
| 25 | This message was deleted |    4 |       2 | 2026-04-09 12:07:33.318364 | 0x01    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 26 | hello guys            |       4 |       2 | 2026-04-09 12:10:28.072918 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 27 | ?hi                   |       4 |       2 | 2026-04-09 12:10:40.684023 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 28 | NULL                  |       4 |       2 | 2026-04-09 12:10:52.988126 | 0x00    | image/png   | Screenshot 2026-03-31 100053.png                          | /uploads/923e93b8-af90-4437-9a6a-89309b065b0c.png         | IMAGE       | DELIVERED |      NULL |
| 29 | NULL                  |       4 |       2 | 2026-04-09 12:11:06.241188 | 0x00    | application/pdf | Assignment for Technical Round Evaluation (48).pdf        | /uploads/d4818e3f-984d-4bcc-a8d1-2ee677230714.pdf         | FILE        | DELIVERED |      NULL |
| 30 | NULL                  |       4 |       2 | 2026-04-09 12:11:13.670546 | 0x00    | audio/webm  | voice-message.webm                                        | /uploads/f2a983b9-085d-4b93-9b3a-a88dbe538a74.webm        | AUDIO       | DELIVERED |      NULL |
| 31 | Hi?                   |       4 |       4 | 2026-04-09 12:12:25.108288 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 33 | NULL                  |       4 |       4 | 2026-04-09 12:12:53.157992 | 0x00    | application/octet-stream | wallllllllllll                                            | /uploads/a174979b-0a34-41bf-bf99-a18c79903744             | FILE        | DELIVERED |      NULL |
| 34 | NULL                  |       4 |       4 | 2026-04-09 12:13:06.482774 | 0x00    | audio/webm  | voice-message.webm                                        | /uploads/d5a64563-9c39-4d80-a2a6-00d6de6d1f3e.webm        | AUDIO       | DELIVERED |      NULL |
| 35 | hello                 |       5 |       4 | 2026-04-09 12:13:25.312473 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 36 | Hi                    |       4 |       2 | 2026-05-09 14:08:49.882518 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 37 | NULL                  |       4 |       2 | 2026-05-09 14:09:08.388054 | 0x00    | image/png   | Screenshot 2026-03-22 121313.png                          | /uploads/1e7521d1-086d-43a3-9a95-8e84e37b2c83.png         | IMAGE       | DELIVERED |      NULL |
| 38 | NULL                  |       4 |       2 | 2026-05-09 14:09:19.357690 | 0x00    | application/pdf | Assignment for Technical Round Evaluation (48).pdf        | /uploads/3e3cfaa94-d81a-4f47-abd5-8d0a98a28a6.pdf         | FILE        | DELIVERED |      NULL |
| 39 | NULL                  |       4 |       2 | 2026-05-09 14:09:44.212576 | 0x00    | audio/webm  | voice-message.webm                                        | /uploads/114753e4-7dc1-4f8c-8965-636ff8d946dd.webm        | AUDIO       | DELIVERED |      NULL |
| 40 | hi guys               |       4 |       4 | 2026-05-09 14:11:43.186692 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 41 | bnzn7V+TZnWUG6HzgvaWVQ== |    4 |      12 | 2026-04-13 12:57:56.452653 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
| 42 | WMxPqIZZ0WECVHB+ueB3g== |     4 |      12 | 2026-04-14 09:38:40.045345 | 0x00    | NULL        | NULL                                                      | NULL                                                      | TEXT        | DELIVERED |      NULL |
+----+-----------------------+---------+---------+----------------------------+---------+-------------+-----------------------------------------------------------+-----------------------------------------------------------+-------------+---------+-----------+
35 rows in set (0.01 sec)
```

```
mysql> select * from chat_room;
+----+-------------+-----------+
| id | room_name   | access_key |
+----+-------------+-----------+
|  1 | general chat | NULL      |
|  8 | Managers    | 6B928C    |
|  9 | Developers  | 8A43DD    |
| 10 | Testing Dept | 359247    |
| 11 | Interns     | 9EE1CD    |
+----+-------------+-----------+
5 rows in set (0.089 sec)

mysql> select * from friendship;
+----+---------------------+---------+-------------+-------------+
| id | created_at          | status  | addressee_id | requester_id |
+----+---------------------+---------+-------------+-------------+
|  1 | 2026-04-09 15:37:41.203844 | ACCEPTED |          3 |           2 |
|  2 | 2026-04-10 11:16:52.868955 | ACCEPTED |          5 |           2 |
+----+---------------------+---------+-------------+-------------+
2 rows in set (0.093 sec)

mysql> select * from message;;
+----+--------------------+-------+-------+-------------+---------+-----------------------+-----------------------------------------------------------+-----------+---------------------+---------+-----------+
| id | content            | room_id | user_id | content_type | deleted | file_name             | file_url                                                  | message_type | sent_at             | status  | recipient_id |
+----+--------------------+-------+-------+-------------+---------+-----------------------+-----------------------------------------------------------+-----------+---------------------+---------+-----------+
|  1 | hey                |     1 |  NULL | NULL        | 0x00    | NULL                  | NULL                                                      | NULL      | NULL                | DELIVERED |      NULL |
|  2 | hello              |     1 |  NULL | NULL        | 0x00    | NULL                  | NULL                                                      | NULL      | NULL                | DELIVERED |      NULL |
|  3 | hy buddies         |     1 |  NULL | NULL        | 0x00    | NULL                  | NULL                                                      | NULL      | NULL                | DELIVERED |      NULL |
|  4 | this is chatroom 1 |     1 |  NULL | NULL        | 0x00    | NULL                  | NULL                                                      | NULL      | NULL                | DELIVERED |      NULL |
|  7 | hello              |     1 |  NULL | NULL        | 0x00    | NULL                  | NULL                                                      | NULL      | NULL                | DELIVERED |      NULL |
|  8 | Hey                |  NULL |     2 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-09 15:01:21.262152 | SENT      |         1 |
|  9 | Vaibhav this side  |  NULL |     2 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-09 15:01:31.317007 | SENT      |         1 |
| 10 | hey vaibhav        |  NULL |     3 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-09 15:40:39.751894 | SENT      |         2 |
| 11 | hi lasya           |  NULL |     2 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-09 15:40:54.333034 | SENT      |         3 |
| 12 | hey all            |     1 |     4 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-09 23:59:07.520338 | DELIVERED |      NULL |
| 13 | hi vaibhav         |  NULL |     2 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-10 09:53:40.071693 | SENT      |         3 |
| 14 | hi vaibhav         |  NULL |     2 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-10 09:53:44.426505 | SENT      |         3 |
| 15 | hello lasya        |  NULL |     3 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-10 09:53:57.075445 | SENT      |         2 |
| 16 | hello vaibhav      |  NULL |     5 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-10 11:17:11.914265 | SENT      |         2 |
| 17 | hey charan         |  NULL |     2 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-10 11:17:29.330695 | SENT      |         5 |
| 18 | iY5jWJY26xqjBFS/2umjTA== |  NULL |     2 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-12 22:33:22.335415 | SENT      |         3 |
| 19 | NULL               |  NULL |     2 | image/png   | 0x00    | Screenshot 2025-12-25 132700.png | /uploads/0e679eed-9281-491e-a7c8-7b6568d53143.png | IMAGE     | 2026-04-12 22:33:56.320019 | SENT      |         3 |
| 20 | dt5h880dZ83BHE1nFScXZQ== |    11 |     2 | NULL        | 0x00    | NULL                  | NULL                                                      | TEXT      | 2026-04-12 22:42:56.075460 | DELIVERED |      NULL |
+----+--------------------+-------+-------+-------------+---------+-----------------------+-----------------------------------------------------------+-----------+---------------------+---------+-----------+
18 rows in set (0.090 sec)
```

```
mysql> select * from message_reaction;
+----+-------+---------------------------+------------+---------+
| id | emoji | reacted_at                | message_id | user_id |
+----+-------+---------------------------+------------+---------+
|  1 | ?     | 2026-04-09 23:59:27.000000 |          8 |       1 |
|  2 | ??    | 2026-04-09 23:59:38.780587 |          2 |       4 |
|  4 | ??    | 2026-04-10 09:54:09.226196 |         15 |       2 |
|  5 | ??    | 2026-04-10 11:17:35.448216 |         17 |       5 |
+----+-------+---------------------------+------------+---------+
4 rows in set (0.078 sec)

mysql> select * from notification;
+----+---------------------------------------+---------------------------+-------------------+---------+
| id | content                               | created_at                | is_read           | user_id |
+----+---------------------------------------+---------------------------+-------------------+---------+
|  1 | Vaibhav sent you a friend request.    | 2026-04-09 15:37:41.303291 | 0x01              |       3 |
|  2 | Lasya accepted your friend request.   | 2026-04-09 15:40:03.510015 | 0x01              |       2 |
|  3 | Vaibhav sent you a friend request.    | 2026-04-10 11:16:52.089960 | 0x00              |       5 |
|  4 | Charan accepted your friend request.  | 2026-04-10 11:16:58.538045 | 0x01              |       2 |
+----+---------------------------------------+---------------------------+-------------------+---------+
4 rows in set (0.159 sec)

mysql> select * from room_access;
+---------+---------+
| room_id | user_id |
+---------+---------+
|       1 |       1 |
|       8 |       2 |
|      11 |       2 |
|       8 |       4 |
|       9 |       4 |
|      10 |       4 |
|      11 |       4 |
+---------+---------+
7 rows in set (0.520 sec)

mysql> select * from users;
+----+----------+-----------------------------+------+---------------------+----------+-------+
| id | username | email                       | otp  | verified            | password | role  |
+----+----------+-----------------------------+------+---------------------+----------+-------+
|  1 | Vaibhav  | NULL                        | NULL | 0x00                | NULL     |       |
|  2 | Vaibhav  | vaibhavbk2124@gmail.com     | NULL | 0x01                | NULL     |       |
|  3 | Ajay     | vaibhavbk.22is@saividya.ac.in | NULL | 0x01                | NULL     |       |
|  4 | admin    | admin@admin.com             | NULL | 0x01                | admin    | ADMIN |
|  5 | Charan   | vaibhavbkbusiness@gmail.com | NULL | 0x01                | NULL     | USER  |
+----+----------+-----------------------------+------+---------------------+----------+-------+
5 rows in set (0.072 sec)
```

```
mysql> select*from notification;
+----+-----------------------------------------+----------------------------+------------------+---------+
| id | content                                 | created_at                 | is_read          | user_id |
+----+-----------------------------------------+----------------------------+------------------+---------+
|  1 | admin sent you a friend request.        | 2026-04-13 12:56:39.998955 | 0x00             |       1 |
|  2 | admin sent you a friend request.        | 2026-04-13 12:56:54.968013 | 0x00             |      11 |
|  3 | admin sent you a friend request.        | 2026-04-14 09:26:56.804514 | 0x01             |       4 |
|  4 | admin sent you a friend request.        | 2026-04-14 09:36:31.869272 | 0x00             |       3 |
|  5 | Keerthana accepted your friend request. | 2026-04-14 09:37:12.837182 | 0x01             |      12 |
|  6 | admin sent you a friend request.        | 2026-04-14 09:37:42.281087 | 0x00             |       2 |
+----+-----------------------------------------+----------------------------+------------------+---------+
6 rows in set (0.01 sec)
```

```
mysql> select*from users;
+----+------------+-----------------------------------+--------+--------------------+----------+-------+
| id | username   | email                             | otp    | verified           | password | role  |
+----+------------+-----------------------------------+--------+--------------------+----------+-------+
|  1 | ram        | NULL                              | NULL   | 0x00               | NULL     |       |
|  2 | kee        | keerthanabhat.3103@gmail.com      | 772922 | 0x00               | NULL     |       |
|  3 | AF05054480 | bhatkeerthana.336@gmail.com       | NULL   | 0x01               | NULL     |       |
|  4 | Keerthana  | keerthanabhat.22is@saividya.ac.in | NULL   | 0x01               | NULL     |       |
|  5 | kee        | NULL                              | NULL   | 0x00               | NULL     |       |
|  6 | kee        | NULL                              | NULL   | 0x00               | NULL     |       |
|  7 | kee123     | NULL                              | NULL   | 0x00               | NULL     |       |
|  8 | kee        | NULL                              | NULL   | 0x00               | NULL     |       |
|  9 | ker        | NULL                              | NULL   | 0x00               | NULL     |       |
| 10 | kee        | NULL                              | NULL   | 0x00               | NULL     |       |
| 11 | ker        | NULL                              | NULL   | 0x00               | NULL     |       |
| 12 | admin      | admin@admin.com                   | NULL   | 0x01               | admin    | ADMIN |
| 13 | AF05054480 | bhatkeerthana336@gmail.com        | NULL   | 0x01               | NULL     | USER  |
+----+------------+-----------------------------------+--------+--------------------+----------+-------+
13 rows in set (0.02 sec)
```

9.3 User Table

Purpose

Stores details of all users who access the system.

Attributes

- id (Primary Key)
- username
- email

Explanation

Each user is uniquely identified by an ID. This table helps track user identity and associate users with messages and chat rooms.

9.4 ChatRoom Table

Purpose

Stores information about chat rooms.

Attributes

- id (Primary Key)
- roomName
- accessKey

Explanation

- Each room has a unique name
- A unique access key is generated for secure entry
- Special case: **general chat does not require a key**

9.5 Message Table

Purpose

Stores all messages sent in chat rooms.

Attributes

- id (Primary Key)
- content (encrypted)
- sentAt
- status
- user_id (Foreign Key)
- room_id (Foreign Key)

Special Feature: Encryption

As described in your document :

- Messages are encrypted using AES before storing
- Only decrypted when displayed

Example

Message (User View)	Stored in DB
Hello	XyZ@#12abc

9.6 Room Access Table

Purpose

Manages access control between users and chat rooms.

Attributes

- room_id
- user_id

Explanation

From your document :

- When user enters correct key → access stored
- Prevents repeated key entry
- Ensures secure room entry

9.7 Relationships Between Tables

- One User → Many Messages
- One ChatRoom → Many Messages
- Many Users ↔ Many Rooms

9.8 Database Screenshot Instructions

 Insert screenshots of:

- User Table
- ChatRoom Table
- Message Table
- Room_Access Table

9.9 Advantages of Database Design

- Organized data storage
- Efficient retrieval
- Secure data handling
- Supports scalability

9.10 Data Integrity Constraints

The database maintains integrity using:

- Primary keys
- Foreign keys
- Constraints

9.11 Normalization

The database is normalized to:

- Reduce redundancy
- Improve efficiency
- Maintain consistency

9.12 Data Security in Database

Data security is a critical component of database management. In this system, message data is not stored in plain text format. Instead, it is encrypted using AES encryption before being stored in the database.

This approach ensures that even if the database is compromised, the attacker cannot understand the stored messages. The encryption process transforms readable text into cipher text, which can only be converted back using the correct key.

This approach ensures that even if the database is compromised, the attacker cannot understand the stored messages. The encryption process transforms readable text into cipher text, which can only be converted back using the correct key.

Additionally, database access is controlled through application-level logic, ensuring that only authorized operations are performed.

9.13 Database Performance Optimization

The performance of the database is optimized through efficient query design and indexing. Primary keys are used to uniquely identify records, while foreign keys establish relationships between tables.

Indexes can be applied to frequently accessed columns such as user ID and room ID to improve query performance. This ensures that messages are retrieved quickly, even as the database grows.

The system avoids redundant data storage by normalizing the database structure, which improves both performance and maintainability.

9.14 Transaction Management

The system ensures that database operations are executed reliably using transaction management. When a message is sent, the system performs multiple operations such as encryption and storage.

If any step fails, the transaction is rolled back to maintain data consistency. This ensures that incomplete or corrupted data is not stored in the database.

9.15 Conclusion

The database design ensures proper data management, security, and efficient communication between different modules of the system.

CHAPTER 10

CODING AND APPLICATION SCREENSHOTS

10.1 Introduction

This chapter explains the coding structure and implementation of the chat application. It includes backend logic, frontend interface, and system functionality.

10.2 Backend Implementation

The backend is developed using Spring Boot, which follows the MVC (Model-View-Controller) architecture.

10.3 Entity Classes

1. Message Entity

Function

Represents chat messages.

Key Feature: AES Encryption

```
@Convert(converterMessageEncryptionConverter.class)
private String content;
```

Explanation

- Automatically encrypts message before storing
- Automatically decrypts when retrieving

2. ChatRoom Entity

Function

Represents chat rooms.

Key Feature: Access Key

- Unique key generated
- Used for secure entry

3. User Entity

Function

Stores user information.

10.4 Controller Classes

1. ChatController

Function

Handles:

- Sending messages
- Loading chat room
- Validating access

2. ChatRoomController

Function

Handles:

- Creating rooms
- Generating access keys

10.5 Service Layer

MessageService

- Saves messages
- Retrieves messages

ChatRoomService

- Manages rooms
- Handles access logic

10.6 Encryption Implementation

Based on your document

Working

1. Message sent
2. Encryption applied
3. Stored in DB
4. Decrypted on display

10.7 Access Control Implementation

Based on your document

Steps

1. Admin creates room
2. Key generated
3. User enters key
4. System verifies
5. Access stored

10.8 Frontend Design

The frontend is built using:

- HTML
- CSS
- JavaScript
- Thymeleaf

10.9 Application Screenshots

 Add screenshots of:

1. Login Page

2. Home Page (Available Rooms)

3. Chat Room Page

4. Enter Key Page

5. Media Upload

10.10 Feature Demonstration

1. Sending Messages

- User types message
- Clicks send
- Message displayed

2. Image Upload

- Select image
- Upload
- Display in chat

3. Audio Message

- Record audio
- Send
- Play in chat

4. File Sharing

- Upload file
- Download option

5. Message Deletion

- Soft delete
- Permanent delete

10.11 Advantages of Implementation

- Secure communication
- Efficient system design
- Modular structure

- Easy maintenance

10.13 MVC Architecture Explanation

The application follows MVC:

Model:

Represents data (User, Message)

View:

UI pages (HTML)

Controller:

Handles logic

10.14 Advantages of MVC

- Separation of concerns
- Easy maintenance
- Better scalability

10.15 Error Handling

The system handles:

- Invalid inputs
- Missing data
- Unauthorized access

10.16 Request-Response Lifecycle

The application follows a request-response lifecycle in which user actions trigger backend processing.

When a user sends a message, the frontend captures the input and sends it to the server through an HTTP request. The controller receives this request and processes it using the service layer.

The service layer performs necessary operations such as validation, encryption, and database interaction. Once the operation is completed, the response is sent back to the frontend, where it is displayed to the user.

This lifecycle ensures smooth communication between the client and server.

10.17 Session Management

Session management is used to track user activity throughout the application. When a user logs in, a session is created and stored on the server.

This session contains user information and is used to authenticate requests. It ensures that only logged-in users can access chat rooms and perform actions.

Sessions are terminated when the user logs out or after a certain period of inactivity, ensuring security.

10.18 File Upload Handling

The system supports file uploads for images, documents, and audio messages. When a file is uploaded, it is stored on the server, and its path is saved in the database.

The system validates the file type and size before uploading to prevent misuse. This ensures that only valid files are stored and displayed.

10.19 Error Handling and Validation

The system includes robust error handling mechanisms to manage unexpected situations. Input validation ensures that users cannot send empty messages or invalid data.

If an error occurs, appropriate messages are displayed to the user, improving usability and reliability.

10.20 Conclusion

The implementation successfully integrates backend logic, frontend design, and database operations to create a fully functional chat system.

CODING

MAIN APPLICATION

ChatappApplication.java

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ChatappApplication {

    public static void main(String[] args) {
        SpringApplication.run(ChatappApplication.class, args);
    }

}
```

SERVICE LAYER

ChatRoomService.java

```
@Service
public class ChatRoomService {

    @Autowired
    private ChatRoomRepository roomRepository;

    @Autowired
    private MessageRepository messageRepository;

    public List<ChatRoom> getAllRooms() {
        return roomRepository.findAll();
    }

    public ChatRoom getRoomById(Long id) {
        return roomRepository.findById(id).orElse(null);
```

```
    }

    public void saveRoom(ChatRoom room) {
        roomRepository.save(room);
    }

    @Transactional
    public void deleteRoomWithMessages(Long id) {
        messageRepository.deleteByRoomId(id);
        roomRepository.deleteById(id);
    }
}
```

EmailService.java

```
@Service
public class EmailService {

    @Autowired
    private JavaMailSender javaMailSender;

    public void sendOtpEmail(String toEmail, String otp) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setTo(toEmail);
        message.setSubject("OTP Verification for Chat Application");
        message.setText("Your OTP is: " + otp);
        javaMailSender.send(message);
    }
}
```

FileStorageService.java

```
@Service
public class FileStorageService {

    @Value("${file.upload-dir}")
    private String uploadDir;

    public String saveFile(MultipartFile file) throws IOException {
        String originalFileName =
StringUtils.cleanPath(file.getOriginalFilename());
        String extension = "";

        int dotIndex = originalFileName.lastIndexOf(".");
        if (dotIndex >= 0) {
            extension = originalFileName.substring(dotIndex);
        }

        String newFileName = UUID.randomUUID() + extension;
```

```
        Path uploadPath = Paths.get(uploadDir).toAbsolutePath().normalize();
        Files.createDirectories(uploadPath);

        Path targetPath = uploadPath.resolve(newFileName);
        Files.copy(file.getInputStream(), targetPath,
StandardCopyOption.REPLACE_EXISTING);

        return "/uploads/" + newFileName;
    }
}
```

MessageService.java

```
@Service
public class MessageService {

    @Autowired
    private MessageRepository messageRepository;

    public List<Message> getMessagesByRoom(Long roomId) {
        return messageRepository.findByRoomIdOrderBySentAtAsc(roomId);
    }

    public List<Message> getDirectMessages(User user1, User user2) {
        return messageRepository.findDirectMessages(user1, user2);
    }

    public void saveMessage(Message message) {
        if (message.getSentAt() == null) {
            message.setSentAt(LocalDateTime.now());
        }
        if (message.getMessageType() == null) {
            message.setMessageType("TEXT");
        }
        if (message.getStatus() == null) {
            message.setStatus("SENT");
        }
        messageRepository.save(message);
    }

    public void softDeleteMessage(Message message) {
        message.setContent("This message was deleted");
        message.setDeleted(true);
        messageRepository.save(message);
    }

    public void permanentlyDeleteMessage(Long messageId) {
        messageRepository.deleteById(messageId);
    }
}
```

NotificationService.java

```
@Service
public class NotificationService {

    @Autowired
    private NotificationRepository notificationRepository;

    public void createNotification(User user, String content) {
        Notification n = new Notification();
        n.setUser(user);
        n.setContent(content);
        notificationRepository.save(n);
    }

    public int getUnreadCount(User user) {
        return notificationRepository.countByUserAndIsReadFalse(user);
    }
}
```

ReactionService.java

```
@Service
public class ReactionService {

    @Autowired
    private MessageReactionRepository reactionRepository;

    public void toggleReaction(Message message, User user, String emoji) {
        Optional<MessageReaction> existing =
            reactionRepository.findByMessageAndUserAndEmoji(message, user,
emoji);

        if (existing.isPresent()) {
            reactionRepository.delete(existing.get());
        } else {
            MessageReaction r = new MessageReaction();
            r.setMessage(message);
            r.setUser(user);
            r.setEmoji(emoji);
            reactionRepository.save(r);
        }
    }
}
```

UserRoomService.java

```
@Service
public class UserRoomService {

    @Autowired
    private UserRoomRepository userRoomRepo;

    public UserRoom save(UserRoom userRoom) {
        return userRoomRepo.save(userRoom);
    }
}
```

UserService.java

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private EmailService emailService;

    public String generateOtp() {
        Random random = new Random();
        return String.valueOf(100000 + random.nextInt(900000));
    }

    public void registerUser(String username, String email) {
        User user = new User();
        user.setUsername(username);
        user.setEmail(email);

        String otp = generateOtp();
        user.setOtp(otp);
        user.setVerified(false);

        userRepository.save(user);
        emailService.sendOtpEmail(email, otp);
    }

    public boolean verifyOtp(String email, String enteredOtp) {
        Optional<User> user = userRepository.findByEmail(email);

        if (user.isPresent() && user.get().getOtp().equals(enteredOtp)) {
            user.get().setVerified(true);
            userRepository.save(user.get());
            return true;
        }
        return false;
```

```
    }
}
```

REPOSITORY LAYER

ChatRoomRepository.java

```
public interface ChatRoomRepository extends JpaRepository<ChatRoom, Long> {
}
```

FriendshipRepository.java

```
@Query("SELECT f FROM Friendship f WHERE (f.requester = :u1 AND f.addressee =
:u2) OR (f.requester = :u2 AND f.addressee = :u1)")
Optional<Friendship> findFriendshipBetween(...);
```

MessageRepository.java

```
List<Message> findByRoomIdOrderBySentAtAsc(Long roomId);
List<Message> findDirectMessages(User user1, User user2);
```

NotificationRepository.java

```
int countByUserAndIsReadFalse(User user);
```

UserRepository.java

```
Optional<User> findByEmail(String email);
```

 Source:

UserRoomRepository.java

```
public interface UserRoomRepository extends JpaRepository<UserRoom, Long> {
}
```

ENTITY CLASSES

ChatRoom.java

```
@Entity
@Table(name = "chat_room")
public class ChatRoom {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String roomName;

    private String accessKey;

    @ManyToMany
    @JoinTable(
        name = "room_access",
        joinColumns = @JoinColumn(name = "room_id"),
        inverseJoinColumns = @JoinColumn(name = "user_id")
    )
    private Set<User> accessedUsers = new HashSet<>();
}
```

Message.java

```
@Entity
@Table(name = "message")
public class Message {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Convert(converter = MessageEncryptionConverter.class)
    private String content;

    private LocalDateTime sentAt;

    private boolean deleted = false;

    private String messageType = "TEXT";

    private String fileName;
    private String fileUrl;
    private String contentType;

    private String status = "SENT";

    @ManyToOne
    private User user;

    @ManyToOne
    private ChatRoom room;

    @ManyToOne
    private User recipient;
}
```

User.java

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;

    @Column(unique = true)
    private String email;

    private String otp;

    private boolean verified;

    private String role = "USER";
```

```
    private String password;
}
```

Notification.java

```
@Entity
public class Notification {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    private User user;

    private String content;

    private boolean isRead = false;
}
```

MessageReaction.java

```
@Entity
public class MessageReaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    private Message message;

    @ManyToOne
    private User user;

    private String emoji;
}
```

UserRoom.java

```
@Entity
@Table(name = "user_room")
public class UserRoom {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    private User user;

    @ManyToOne
    private ChatRoom room;
}
```

Friendship.java

```
@Entity
public class Friendship {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    private User requester;

    @ManyToOne
    private User addressee;

    private String status = "PENDING";
}
```

CONTROLLER LAYER

AuthController.java

```
@PostMapping("/sendOtp")
public String sendOtp(@RequestParam String username,
@RequestParam String email,
Model model) {

    userService.registerUser(username, email);
    model.addAttribute("email", email);
    return "verify-otp";
}
```

ChatController.java

```
@GetMapping("/room/{id}")
public String openRoom(@PathVariable Long id, Model model, HttpSession
session) {

    User loggedInUser = (User) session.getAttribute("loggedInUser");

    if (loggedInUser == null) {
        return "redirect:/login";
    }

    ChatRoom room = roomService.getRoomById(id);

    if (!room.getAccessedUsers().contains(loggedInUser)) {
        return "enter-key";
    }

    model.addAttribute("messages", messageService.getMessagesByRoom(id));
    return "chat";
}
```

ChatRoomController.java

```
@PostMapping("/addRoom")
public String addRoom(@RequestParam String roomName, HttpSession session) {

    ChatRoom room = new ChatRoom();
    room.setRoomName(roomName);
    room.setAccessKey(UUID.randomUUID().toString().substring(0,
6).toUpperCase());

    roomService.saveRoom(room);

    return "redirect:/";
}
```

FriendshipController.java

```
@PostMapping("/add/{id}")
public String addFriend(@PathVariable Long id, HttpSession session) {
    friendshipService.sendRequest(loggedInUser, addressee);
}
```

NotificationController.java

```
@PostMapping("/markRead")
public String markAllAsRead(HttpSession session) {
    notificationService.markAllAsRead(loggedInUser);
}
```

ReactionController.java

```
@PostMapping("/toggle")
public String toggleReaction(@RequestParam Long messageId,
                             @RequestParam String emoji) {
    reactionService.toggleReaction(message, user, emoji);
}
```

UserController.java

```
@PostMapping("/addUser")
public String addUser(@RequestParam String username) {
    userService.saveUser(user);
}
```

CONFIGURATION

FileUploadConfig.java

```
registry.addResourceHandler("/uploads/**")
        .addResourceLocations(uploadPath);
```

MessageEncryptionConverter.java

```
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
cipher.init(Cipher.ENCRYPT_MODE, secretKey);
```

CONFIGURATION FILE

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/chatdb
spring.datasource.username=root
spring.datasource.password=vaibhav

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.thymeleaf.cache=false

spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=vaibhavbk2124@gmail.com
spring.mail.password=vpeabqxcwabfppnc
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

file.upload-dir=uploads

spring.servlet.multipart.max-file-size=20MB
spring.servlet.multipart.max-request-size=25MB

spring.mail.properties.mail.smtp.ssl.trust=smtp.gmail.com

server.port=8082
```

TEST FILE

ChatappApplicationTests.java

```
package com.example;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
```

```
@SpringBootTest
class                        ChatappApplicationTests                        {

    @Test
    void                              contextLoads()                         {
    }

}
```

FRONTEND

---

login.html

```
<form                      action="/sendOtp"                      method="post">
    <input            type="text"            name="username"            required>
    <input            type="email"             name="email"             required>
    <button                   type="submit">Send                   OTP</button>
</form>
```

---

verify-otp.html

```
<form                     action="/verifyOtp"                     method="post">
    <input        type="hidden"        name="email"        th:value="${email}">
    <input             type="text"             name="otp"             required>
    <button                  type="submit">Verify                  OTP</button>
</form>
```

---

admin-login.html

```
<form                    action="/adminLogin"                    method="post">
    <input            type="text"            name="username"            required>
    <input          type="password"          name="password"          required>
    <button                 type="submit">Secure                 Login</button>
</form>
```

 Source:

---

index.html

```
<h1>Chat                                                                Rooms</h1>

<a                                                     href="/logout">Logout</a>

<div                 th:each="room                 :                 ${rooms}">
    <a             th:href="@{'/room/'             +             ${room.id}}">
        <span                                th:text="${room.roomName}"></span>
    </a>
</div>
```

---

enter-key.html

```
<form                 th:action="@{/room/join}"                 method="post">
    <input       type="hidden"       name="roomId"       th:value="${roomId}">
    <input           type="text"           name="accessKey"           required>
    <button                                         type="submit">Join</button>
</form>
```

---

chat.html

```
<div                 th:each="m                 :                 ${messages}">
    <div                                          th:text="${m.content}"></div>

    <div            th:if="${m.messageType            ==            'IMAGE'}">
        <img                                            th:src="${m.fileUrl}">
    </div>

    <div            th:if="${m.messageType            ==            'AUDIO'}">
        <audio                                                       controls>
            <source                                     th:src="${m.fileUrl}">
        </audio>
    </div>

    <div            th:if="${m.messageType            ==            'FILE'}">
        <a         th:href="${m.fileUrl}"         th:download="${m.fileName}">
            Download                                                      File
        </a>
    </div>
</div>

<form                       action="/send"                       method="post">
    <input                      type="text"                     name="content">
    <button                                         type="submit">Send</button>
</form>
```

---

direct-chat.html

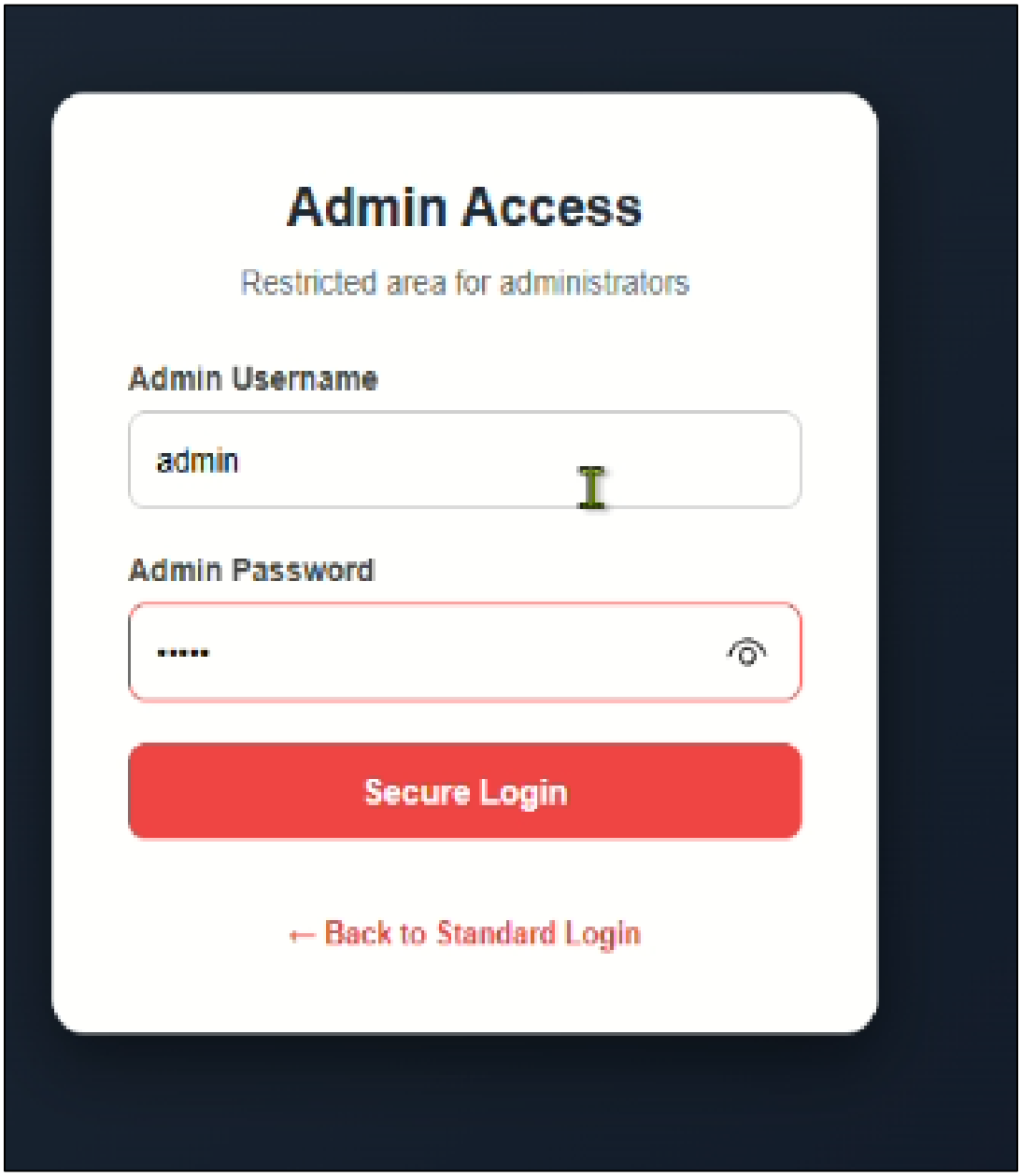
```
<form                    action="/send-direct"                    method="post">
    <input                       type="text"                     name="content">
    <button                                                 type="submit">Send</button>
</form>
```

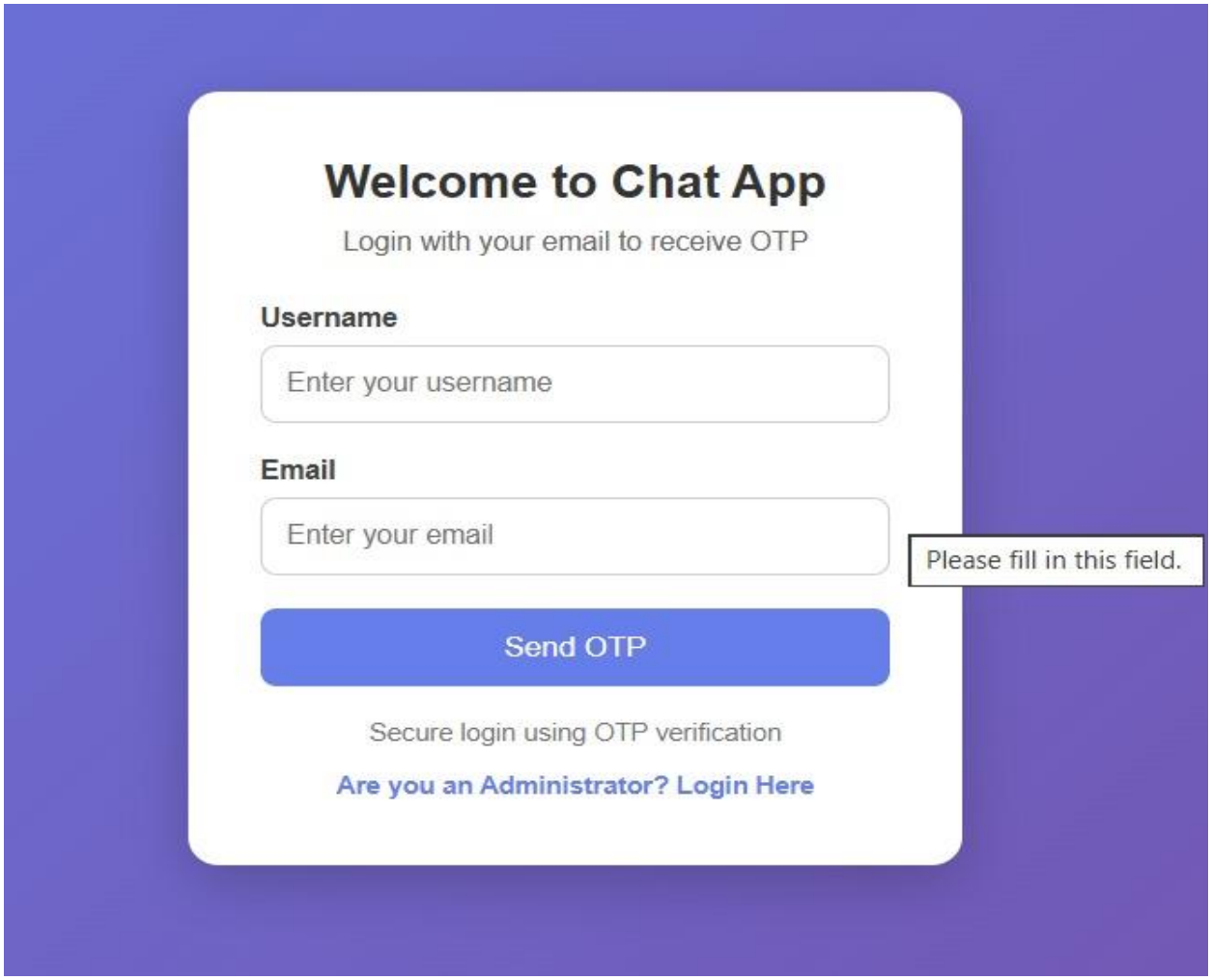
CSS FILE

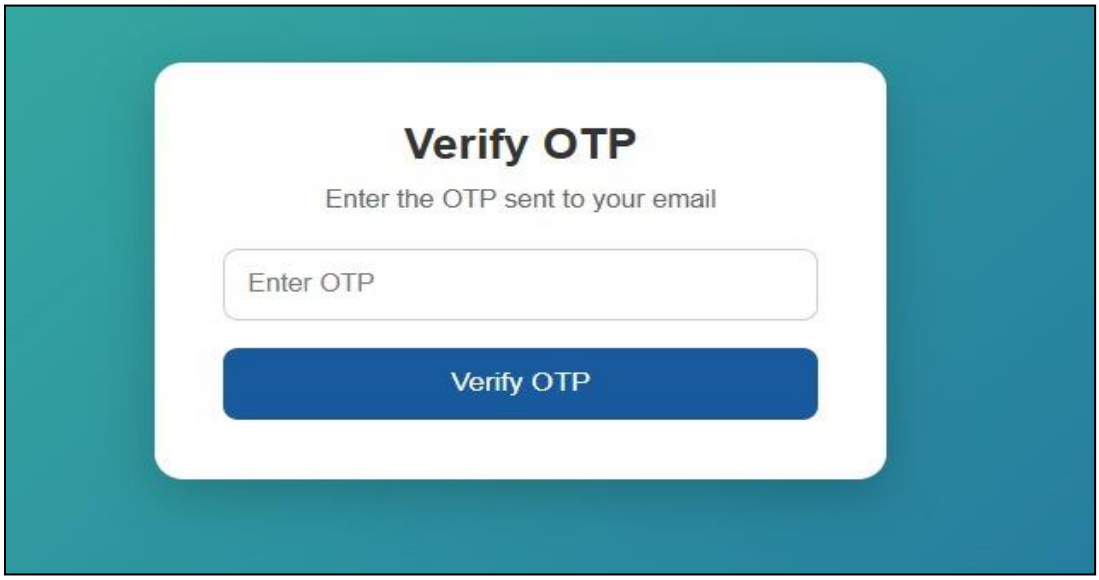
style.css

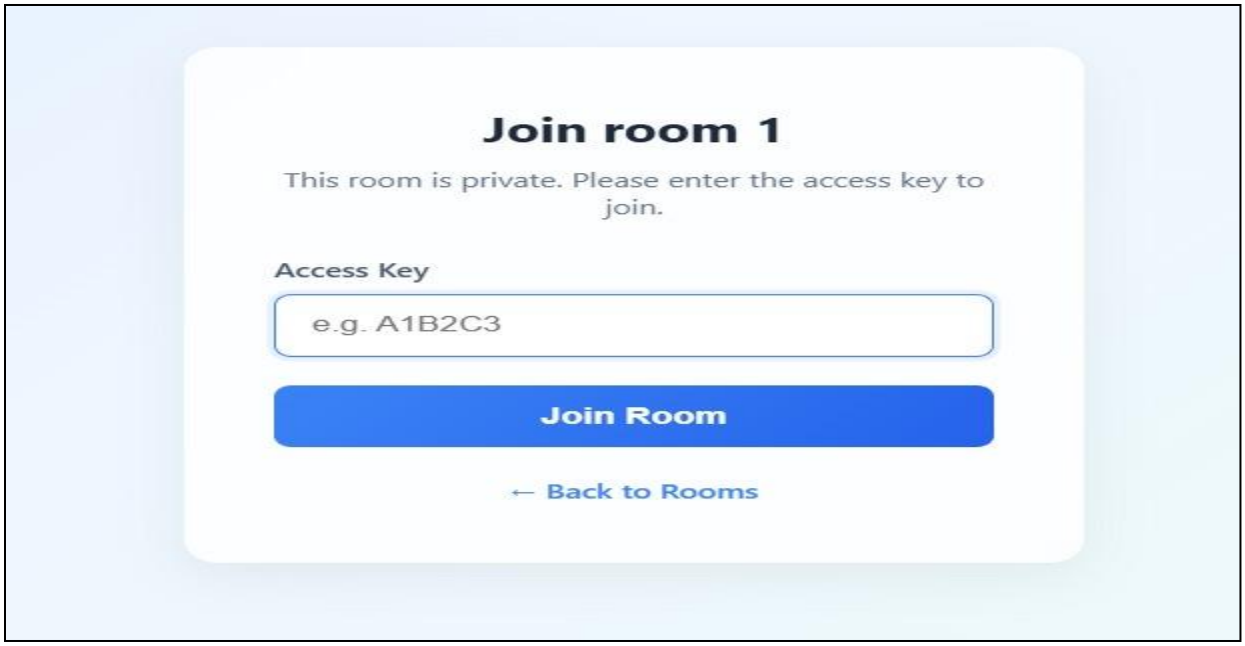
```
body                                                                        {
    font-family:                           "Segoe                          UI";
    background:        linear-gradient(135deg,        #dbeafe,        #ecfdf5);
}

.room-item                                                                  {
    background:                                                      #f8fbff;
    border-radius:                                                      18px;
}
```

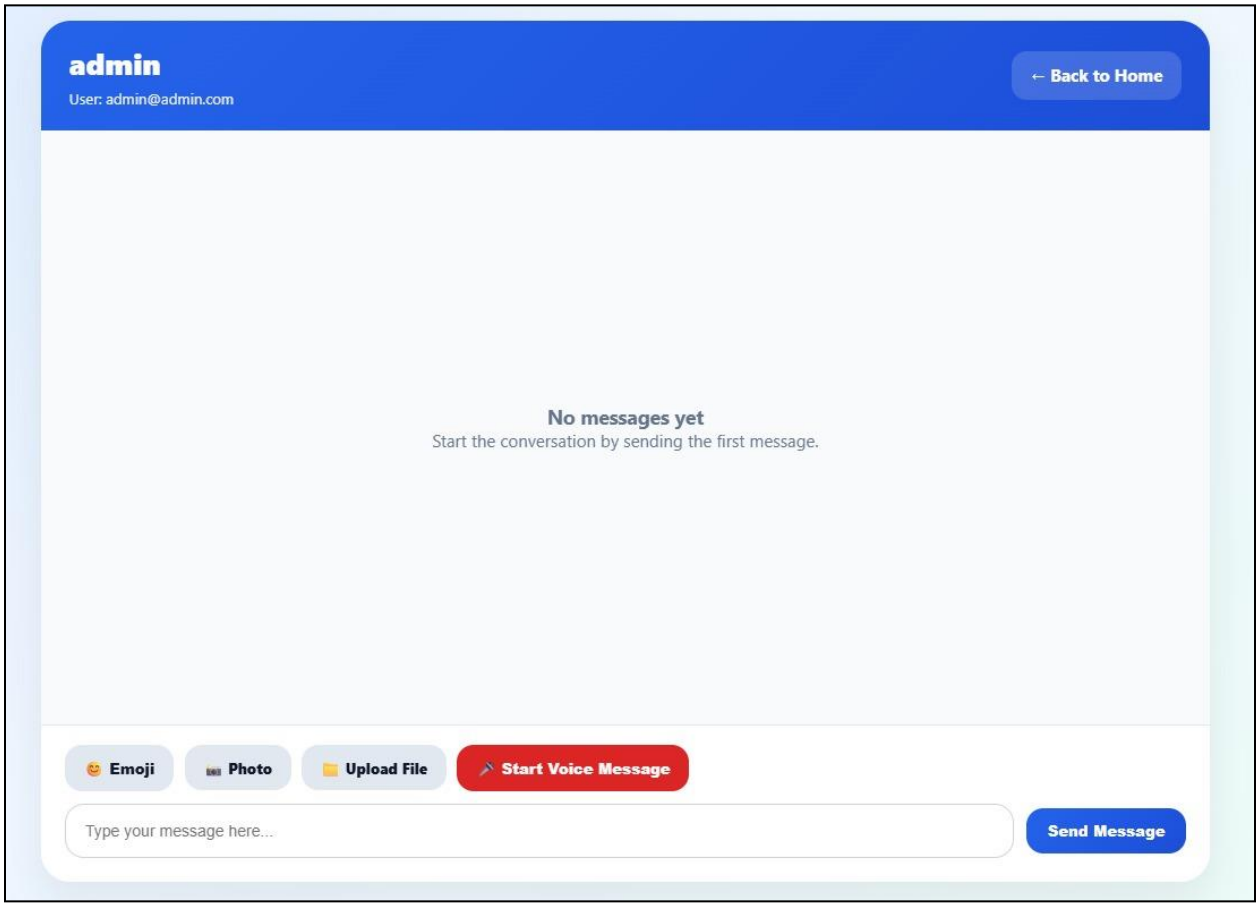
SCREENSHOTS OF THE APPLICATION

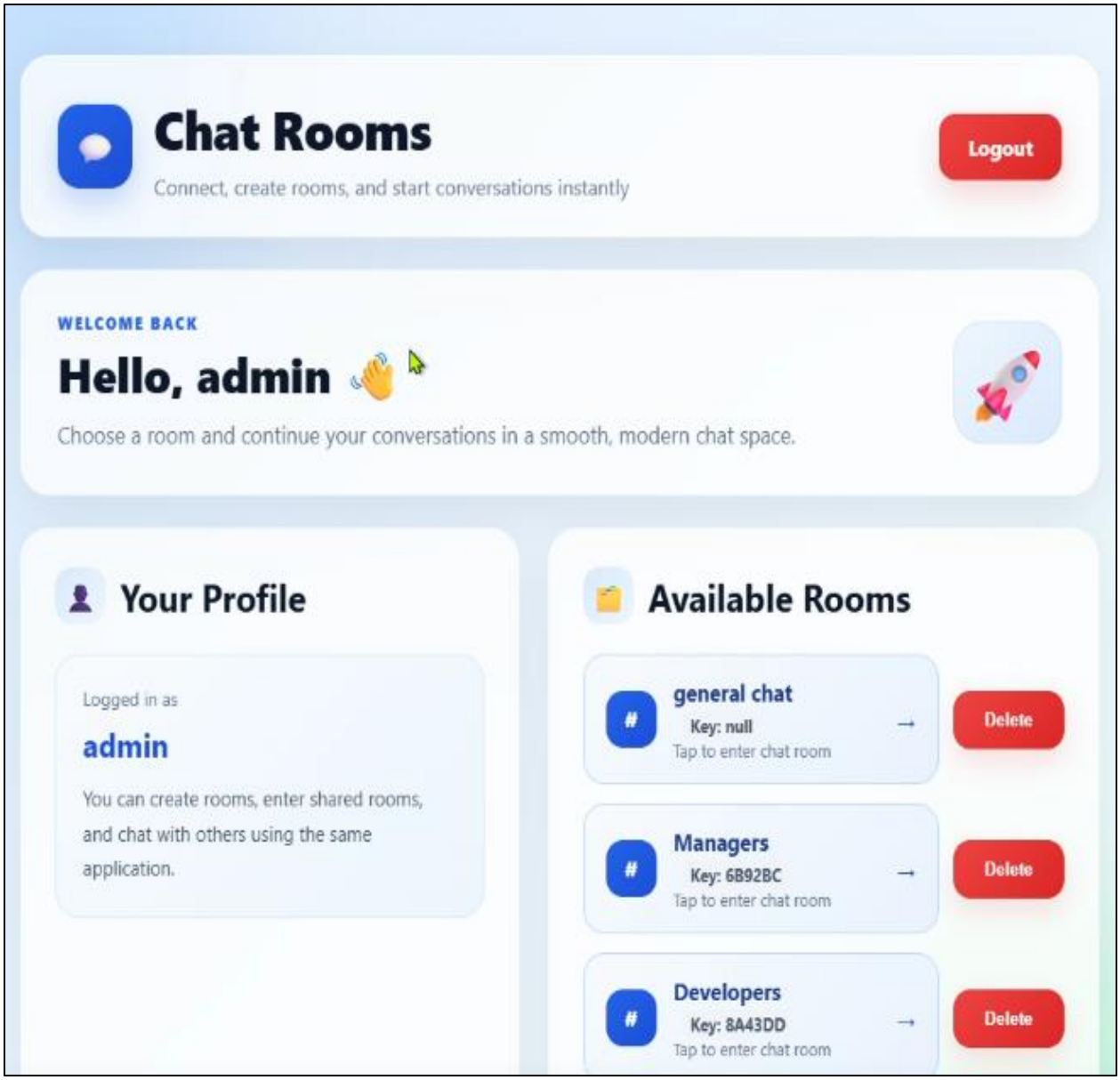


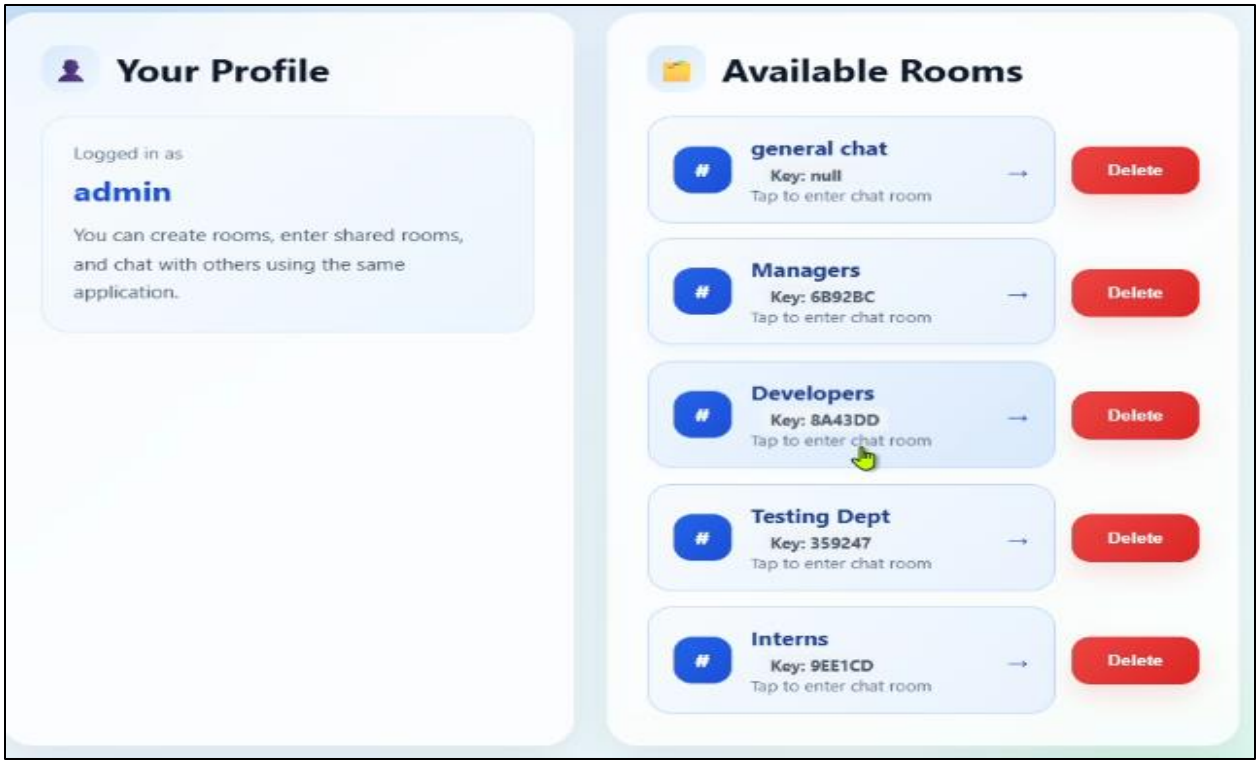

Welcome to Chat App
Login with your email to receive OTP
Username
Enter your username
Email
Enter your email
Please fill in this field.
Send OTP
Secure login using OTP verification
Are you an Administrator? Login Here

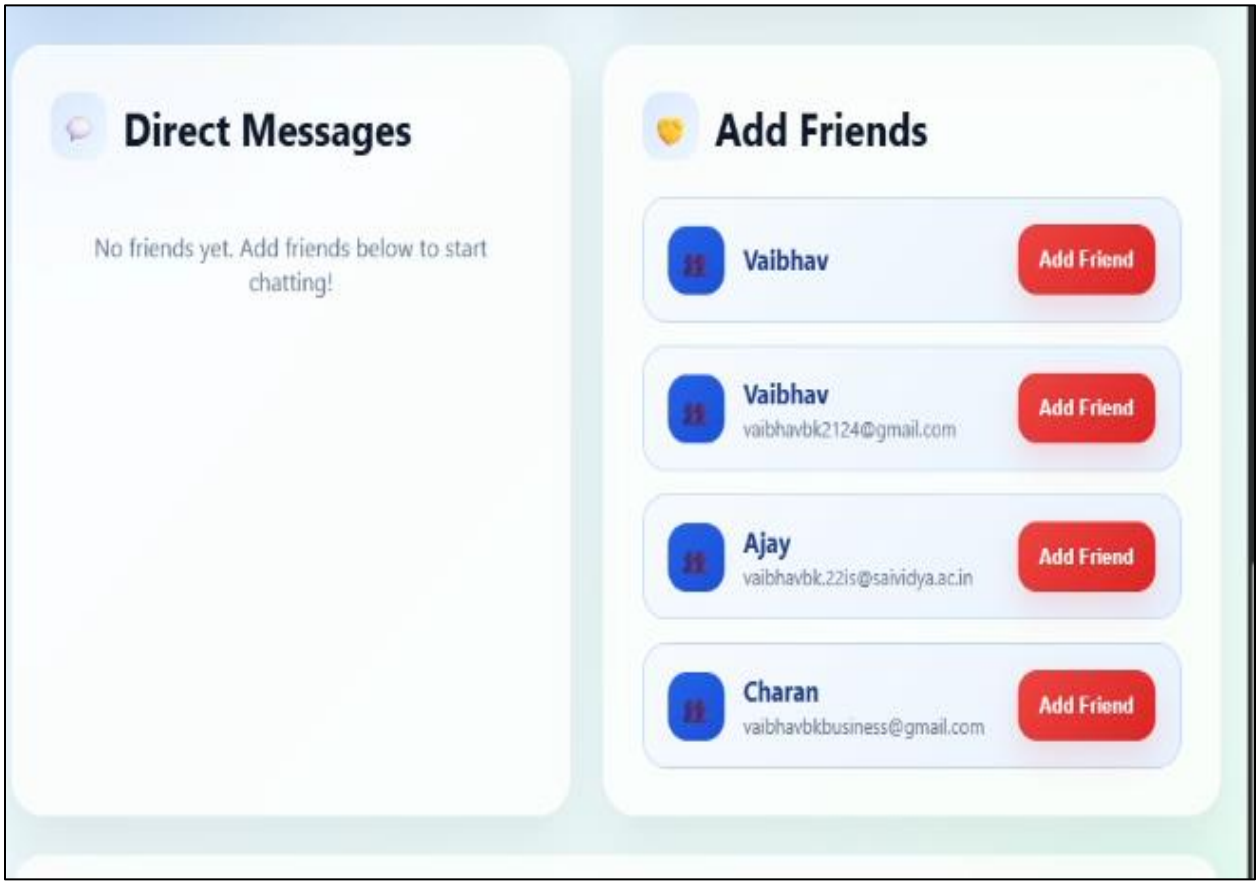

Verify OTP
Enter the OTP sent to your email
Enter OTP
Verify OTP

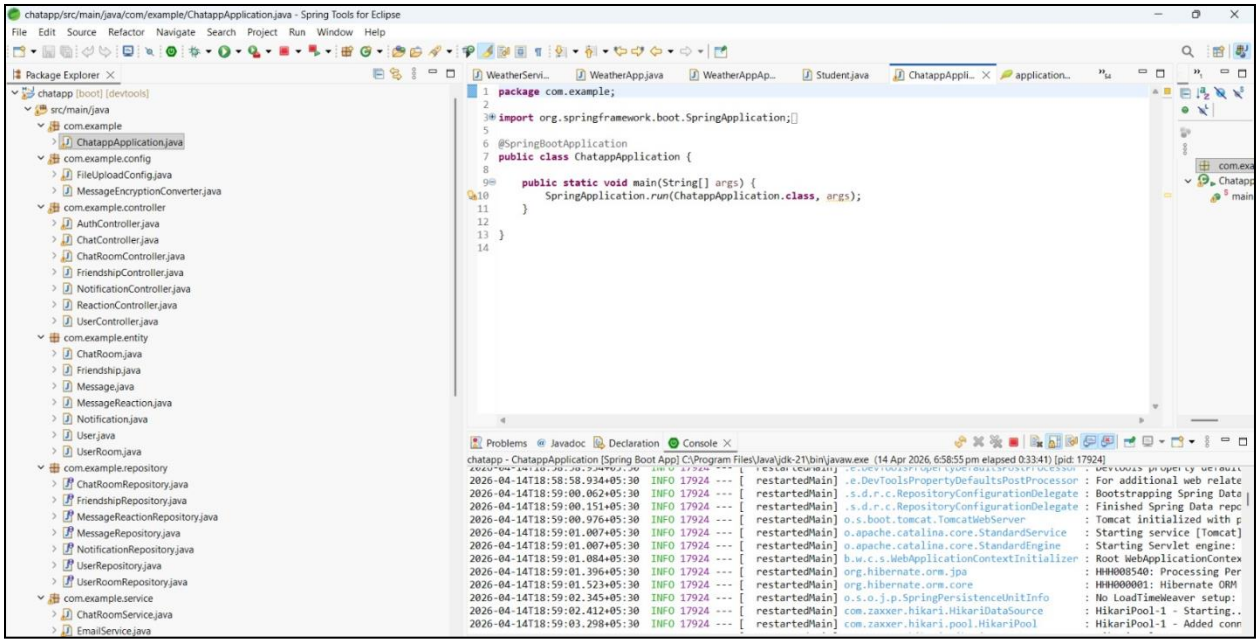

Join room 1
This room is private. Please enter the access key to join.
Access Key
e.g. A1B2C3
Join Room
← Back to Rooms

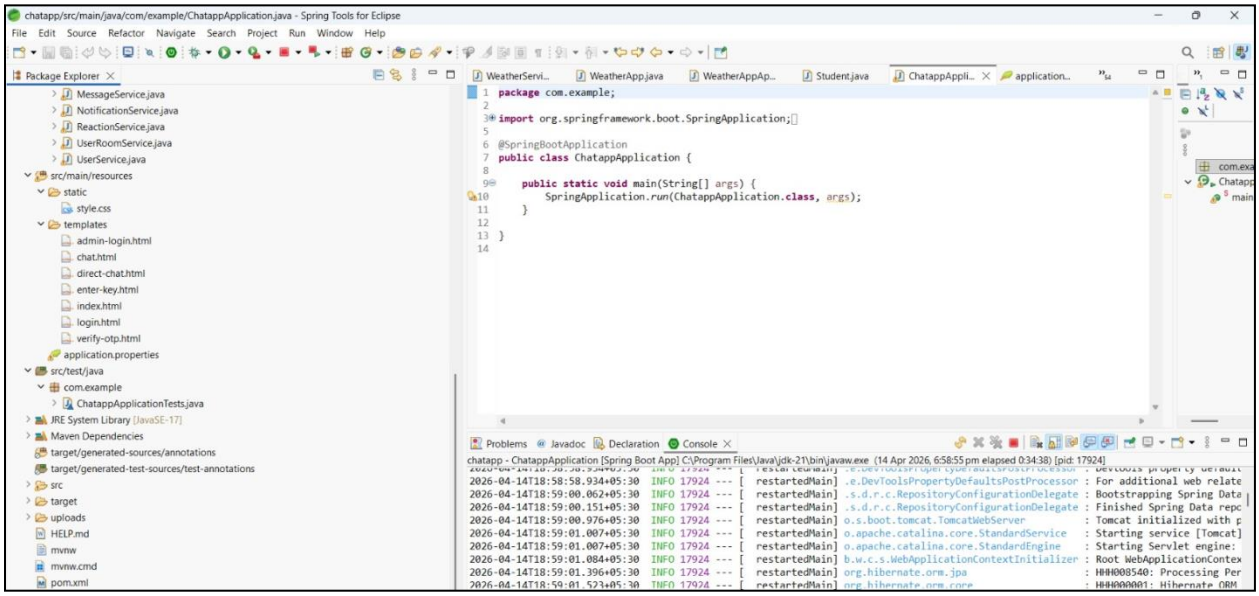

admin
User: admin@admin.com
← Back to Home
No messages yet
Start the conversation by sending the first message.
Emoji
Photo
Upload File
Start Voice Message
Type your message here...
Send Message


Chat Rooms
Connect, create rooms, and start conversations instantly
Logout
WELCOME BACK
Hello, admin
Choose a room and continue your conversations in a smooth, modern chat space.
Your Profile
Logged in as
admin
You can create rooms, enter shared rooms, and chat with others using the same application.
Available Rooms
general chat
Key: null
Tap to enter chat room
Delete
Managers
Key: 6B92BC
Tap to enter chat room
Delete
Developers
Key: 8A43DD
Tap to enter chat room
Delete


Your Profile
Logged in as
admin
You can create rooms, enter shared rooms, and chat with others using the same application.
Available Rooms
general chat
Key: null
Tap to enter chat room
Delete
Managers
Key: 6B92BC
Tap to enter chat room
Delete
Developers
Key: 8A43DD
Tap to enter chat room
Delete
Testing Dept
Key: 359247
Tap to enter chat room
Delete
Interns
Key: 9EE1CD
Tap to enter chat room
Delete


Direct Messages
No friends yet. Add friends below to start chatting!
Add Friends
Vaibhav
Add Friend
Vaibhav
vaibhavbk2124@gmail.com
Add Friend
Ajay
vaibhavbk.22is@saividya.ac.in
Add Friend
Charan
vaibhavbkbusiness@gmail.com
Add Friend


chatapp/src/main/java/com/example/ChatappApplication.java - Spring Tools for Eclipse
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
chatapp [boot] [devtools]
src/main/java
com.example
ChatappApplication.java
com.example.config
FileUploadConfig.java
MessageEncryptionConverter.java
com.example.controller
AuthController.java
ChatController.java
ChatRoomController.java
FriendshipController.java
NotificationController.java
ReactionController.java
UserController.java
com.example.entity
ChatRoom.java
Friendship.java
Message.java
MessageReaction.java
Notification.java
User.java
UserRoom.java
com.example.repository
ChatRoomRepository.java
FriendshipRepository.java
MessageReactionRepository.java
MessageRepository.java
NotificationRepository.java
UserRepository.java
UserRoomRepository.java
com.example.service
ChatRoomService.java
EmailService.java
WeatherServi... WeatherApp.java WeatherAppAp... Student.java ChatappAppli... application...
package com.example;
import org.springframework.boot.SpringApplication;
@SpringBootApplication
public class ChatappApplication {
public static void main(String[] args) {
SpringApplication.run(ChatappApplication.class, args);
}
}
Problems Javadoc Declaration Console
chatapp - ChatappApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (14 Apr 2026, 6:58:55 pm elapsed 0:33:41) [pid: 17924]
2026-04-14T18:58:58.934+05:30 INFO 17924 --- [restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web relate
2026-04-14T18:59:00.062+05:30 INFO 17924 --- [restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data
2026-04-14T18:59:00.151+05:30 INFO 17924 --- [restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repo
2026-04-14T18:59:00.976+05:30 INFO 17924 --- [restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with p
2026-04-14T18:59:01.007+05:30 INFO 17924 --- [restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-04-14T18:59:01.007+05:30 INFO 17924 --- [restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine:
2026-04-14T18:59:01.084+05:30 INFO 17924 --- [restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContex
2026-04-14T18:59:01.396+05:30 INFO 17924 --- [restartedMain] org.hibernate.orm.jpa : HHH008540: Processing Per
2026-04-14T18:59:01.523+05:30 INFO 17924 --- [restartedMain] org.hibernate.orm.core : HHH000001: Hibernate ORM
2026-04-14T18:59:02.345+05:30 INFO 17924 --- [restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup:
2026-04-14T18:59:02.412+05:30 INFO 17924 --- [restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting..
2026-04-14T18:59:03.298+05:30 INFO 17924 --- [restartedMain] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added conn


chatapp/src/main/java/com/example/ChatappApplication.java - Spring Tools for Eclipse
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
MessageService.java
NotificationService.java
ReactionService.java
UserRoomService.java
UserService.java
src/main/resources
static
style.css
templates
admin-login.html
chat.html
direct-chat.html
enter-key.html
index.html
login.html
verify-otp.html
application.properties
src/test/java
com.example
ChatappApplicationTests.java
JRE System Library [JavaSE-17]
Maven Dependencies
target/generated-sources/annotations
target/generated-test-sources/test-annotations
src
target
uploads
HELP.md
mvnw
mvnw.cmd
pom.xml
WeatherServi... WeatherApp.java WeatherAppAp... Student.java ChatappAppli... application...
package com.example;
import org.springframework.boot.SpringApplication;
@SpringBootApplication
public class ChatappApplication {
public static void main(String[] args) {
SpringApplication.run(ChatappApplication.class, args);
}
}
Problems Javadoc Declaration Console
chatapp - ChatappApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (14 Apr 2026, 6:58:55 pm elapsed 0:34:38) [pid: 17924]
2026-04-14T18:58:58.934+05:30 INFO 17924 --- [restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web relate
2026-04-14T18:59:00.062+05:30 INFO 17924 --- [restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data
2026-04-14T18:59:00.151+05:30 INFO 17924 --- [restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repo
2026-04-14T18:59:00.976+05:30 INFO 17924 --- [restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with p
2026-04-14T18:59:01.007+05:30 INFO 17924 --- [restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-04-14T18:59:01.007+05:30 INFO 17924 --- [restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine:
2026-04-14T18:59:01.084+05:30 INFO 17924 --- [restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContex
2026-04-14T18:59:01.396+05:30 INFO 17924 --- [restartedMain] org.hibernate.orm.jpa : HHH008540: Processing Per

CHAPTER 11

CONCLUSION

11.1 Summary of the Project

The Room-Based Chat Application successfully demonstrates the development of a web-based communication system using modern technologies such as Spring Boot and MySQL.

The system enables users to:

- Communicate through chat rooms
- Share multimedia
- Access rooms securely
- Send encrypted messages

11.2 Key Achievements

- Implementation of AES encryption
- Secure room access using keys
- Multimedia communication
- User-friendly interface

11.3 Learning Outcomes

Through this project, the following concepts were learned:

- Spring Boot framework
- Database design
- Encryption algorithms
- Web development

- System design principles

11.4 Future Enhancements

- WebSocket for real-time chat
- Mobile application
- Push notifications
- Emoji support

11.5 Final Conclusion

The project meets all its objectives and provides a strong foundation for developing advanced communication systems. It successfully demonstrates the integration of security, database management, and user interaction in a web-based application.

11.6 Practical Applications

- Online classrooms
- Team collaboration
- Project discussions

11.7 Project Impact

The project demonstrates how secure communication systems can be built using modern technologies. It provides practical knowledge and enhances understanding of real-world applications.